

Experiment-Overview 2026-05-31

Experiment-Overview: TDD-Workflows × Modelle × Prompt-Stile

Stand: 2026-05-31. Datenbasis: `experiments/runs/` (921 Runs gesamt).

Autor: Marco Emrich (codecentric AG) — Mit-Initiator von [EXACT Coding](#) gemeinsam mit Ferdinand Ade.

Repository: github.com/marcoemrich/agentive_coding_lab — alle Skripte, Workflow-Definitionen, Run-Artefakte und das Stylesheet sind dort öffentlich versioniert.

Über die Studie

Dieses Lab ist die empirische Validierungs-Plattform für **EXACT Coding** (EXample-guided AI-Collaborative Test-driven Coding) — den Workflow-Ansatz, der im Manuskript unter `../../../../exact-coding-book/manuscript/exact-coding.md` (relativ zur Repo-Wurzel) entwickelt wird. EXACT Coding verbindet beispielgeführte Spezifikation (Example Mapping) mit striktem Test-Driven Development und einer expliziten Refactor-Disziplin. Die untersuchten Workflow-Varianten spannen ein Spektrum auf: von Vibe-Coding-Baselines ohne Tests (ein Agent schreibt Code in einem Schritt) über EXACT-konforme Setups mit Test-Liste, Red/Green/Refactor-Zyklen und phasen-isolierten Subagents bis zu einer Delayed-Refactor-Kontrolle (Oneshot-Implementation → nachträgliche Tests → einmaliger End-Refactor). Diese Achse erlaubt es, jeden EXACT-Baustein — beispielgeführte Spec, Test-First, periodisches Refactoring, Kontext-Isolation — einzeln gegen seine Abwesenheit zu messen.

Dieser Snapshot fasst den Stand vom 2026-05-31 zusammen: 921 Runs über 26 Forschungsfragen, verteilt auf drei Harness-Subtrees (Claude Code, OpenCode, harness-übergreifend) sowie eine Workflow-Entwicklungs-Linie. Die aktuelle Forschungs-Front liegt auf zwei Themen: erstens der Frage, ob ein metric-driven Refactor-Pass (der ESLint- und Komplexitäts-Metriken selbst pre/post misst) die Code-Qualität gegenüber einem Per-Cycle-Refactor verbessert — geprüft sowohl auf Opus 4.7 als auch auf dem neuen Opus 4.8; zweitens dem Modell-Vergleich über Nicht-Anthropic-Modelle hinweg via OpenCode-Harness. Der korrektheits-orientierte Default für komplexe, novel Aufgaben ist derzeit ein Hybrid-Workflow mit Skill-basiertem Red/Green im geteilten Kontext und isoliertem Refactor-Subagent, ergänzt um kausale Why-Begründungen neben den imperativen Anweisungen.

Scope

Die Befunde gelten für einen eng definierten Drei-Achsen-Scope. **(1) Harness:** Der Kern der Studie läuft auf der **Claude-Code-CLI** (Container-Pin `claude-code@2.1.156`, siehe `experiments/docker/Dockerfile`), headless und ohne Human-in-the-Loop. Eine Teilmenge der RQs erweitert bewusst auf die Harnesses **OpenCode** und **pi**, um Harness-Effekte zu isolieren — diese sind als solche gekennzeichnet (Subtrees `questions-opencode/`, `questions-cross/`). **(2) Modelle:** Der Claude-Code-Kern nutzt ausschließlich **Anthropic-Modelle** (Opus 4.6/4.7/4.8, Sonnet 4.6, Haiku 4.5 — jeweils mit/ohne Thinking, Direct-API und Portkey-Routing). Die OpenCode-RQs ziehen zusätzlich Nicht-Anthropic-Modelle (GLM, Gemini, Kimi, DeepSeek, MiniMax, Mistral) heran — auch das ist explizit als Harness-/Modell-Erweiterung markiert. **(3) Zielsprache:** ausschließlich **TypeScript** mit einem festen `pnpm/tsx/vitest/ESLint+SonarJS`-Stack pro Run. Der Transfer auf andere agentische Tools (Cursor, Aider, Cline, Windsurf), andere Provider in einem Anthropic-nativen Setup, andere Zielsprachen (Python, Go, Java) oder interaktive HITL-Setups ist offen und liegt außerhalb dieses Scopes.

AI-Hinweis

Dieser Snapshot wurde mit der `/build-overview`-Skill in **Claude Code** (Anthropic Opus 4.8) erstellt. Datengetriebene Sektionen — RQ-Übersichts-Tabelle, Coverage-Werte, Finding-Listen pro RQ, Reproduzierbarkeits- und Files-Tabelle — werden deterministisch aus `research/{questions,workflow-dev}/*/README,findings.md` via `experiments/generate-snapshot-skeleton.py` generiert. Synthese-Sektionen (Intro, Per-RQ-Paragraphen, Cross-RQ-Synthese, Limitierungen) sind vom LLM gedraftet und human-curated. Die Generierung ist damit vollständig nachvollziehbar.

Hauptbefunde

Acht zentrale Befunde aus den 26 Forschungsfragen — Details und Belege in §4, Cross-RQ-Synthese in §5:

- EXACT-Coding wirkt — die Kombo aus Example-Mapping und Tests-gegen-Spec schlägt Vibe-Coding messbar.** Auf der novel Kata (claim-office) fällt die Korrektheit (außen, `verification_pct`) von ≥ 0.96 auf 0.28, sobald ohne Test-Schreib-Phase geübt wird; Example-Mapping als Spec-Stil hebt sie zusätzlich um +48–64 Prozentpunkte gegenüber Prose. Beide Korrektheits-Hebel sind die *Spezifikation* (konkrete I/O-Beispiele) und das *Formulieren als Tests gegen die Spec* — nicht der Red-Green-Refactor-Zyklus selbst (der naive „use TDD“-Lauf erreicht ebenfalls 1.00 Korrektheit). Praktische Konsequenz: Auf novel Domänen sind konkrete I/O-Beispiele plus eine Test-Schreib-Phase die wertvollste Korrektheits-Investition.
- Striktes TDD verbessert die Code-Qualität messbar.** Ein Workflow mit periodischem, isoliertem Refactor-Schritt senkt auf claim-office die Komplexitäts-Spitze auf $\sim \frac{1}{3}$ und die Smell-Summe auf $\sim \frac{1}{10}$ von Vibe-Coding (`cognitive_max` 5.7 vs. 11–12, `smell_total` 1.3 vs. 12–16). Den Hebel liefert die strukturierte Refactor-Disziplin im Takt, nicht das Etikett „TDD“. Der naive Ansatz — ein Agent, der nur „use TDD“ hört und ohne erzwungenen Red-Green-Refactor-Takt sich selbst überlassen bleibt — produziert den schwersten Code der ganzen Matrix (`cognitive_max` 19.8), schlechter als gar kein TDD. Praktische Konsequenz: Für langlebigen Code zahlt sich ein Workflow mit erzwungenem Aufräum-Schritt pro Zyklus aus; eine bloße „mach es in TDD“-Aufforderung tut es nicht.
- Example-Mapping ist auf novel Aufgaben der dominante Korrektheits-Hebel — User-Story = Prose.** Auf claim-office hebt Example-Mapping `verification_pct` um +48–64 Prozentpunkte gegenüber Prose (Opus 4.6 no-thinking: 0.23 → 0.87), weil konkrete Input/Output-Beispiele die Domänen-Mehrdeutigkeiten auflösen. User-Story wirkt praktisch identisch zu Prose ($\Delta \leq 6$ pp). Auf trainingsbekannten Katas ist der Effekt null. Praktische Konsequenz: Beim Schreiben einer Spec für eine novel Domäne sind konkrete I/O-Paare die wertvollste Investition.

4. Eine Test-Schreib-Phase ist die billigste Korrektheits-Versicherung — Position egal. Auf claim-office kollabiert reines Vibe-Coding auf 28 % (4/15 Szenarien); sobald irgendeine Phase Tests gegen die Spec schreibt — auch ein *nachträglicher* Test-Pass nach Oneshot-Implementation — springt die Korrektheit auf ≥ 96 %. Praktische Konsequenz: Niemals testfrei auf ein neues Problem; die nachgelagerte Test-Phase aus der Delayed-Refactor-Kontrolle reicht bereits, um Vibe-Coding korrektkeits-sicher zu machen.
5. Ein Hybrid-Workflow mit Skill-basiertem Red/Green im geteilten Kontext und isoliertem Refactor-Subagent ist der robuste Code-Qualitäts-Default — aber kein Workflow gewinnt universell. Diese Architektur landet als einzige über beide Code-Qualitäts-Katas in den Top-2 (*cognitive_max* 5.7 auf claim-office vs. 19.8 bei Minimal-TDD) und ist die einzige mit 0 % Outlier-Rate. Zugleich tauschen phasen-isolierte und hybride Workflows je nach Modell die Plätze. Praktische Konsequenz: Refactor-Phase plus Refactor-Isolation sind die Hebel — aber der beste Workflow ist modellabhängig zu validieren.
6. Code-Qualität kostet eine Größenordnung mehr Tokens — nach Code-Lebensdauer wählen. Strikte TDD-Workflows verbrauchen auf claim-office bis zu $\sim 16\times$ mehr Tokens als Vibe-Coding mit einmaligem End-Refactor, bei *gleicher* Korrektheit; der Mehrwert ist Verzweigungs-Komplexität (*cognitive_max* 5.7 vs. 7.4–11.0), nicht Korrektheit. Praktische Konsequenz: für langlebigen Produktiv-Code lohnt der Aufpreis, für Wegwerf-Code ist die günstige Delayed-Refactor-Variante die rationale Wahl.
7. Kausale Why-Begründungen neben imperativen Anweisungen verbessern Disziplin und Code-Qualität deutlich, ohne die Korrektheit zu berühren. Die Form “MUST X. Why: Y.” senkt auf claim-office *cognitive_max* um -43 %, Smells um -87 % und die Streuung der Komplexitäts-Spitzen um 82 – 90 %, bei invarianter Korrektheit und $+22$ % Tokens. Praktische Konsequenz: Begründungen sind kein Prompt-Bloat, sondern ein wirksamer Qualitäts-Hebel.
8. Prompt-Änderungen müssen auf einer novel Kata gegengeprüft werden — game-of-life allein verbirgt Brüche. Mehrere additive Workflow-Erweiterungen (Audit-Bundle, Refactor-Vokabular) sind auf der trainingsbekannten Kata neutral bis leicht positiv, brechen aber auf claim-office die Korrektheit von 0.96 auf 0.23 – 0.35 ein, weil der Agent sich vorzeitig fertig erklärt. Praktische Konsequenz: jede Workflow-/Prompt-Änderung gegen eine novel Kata mit echten Mehrdeutigkeiten testen, bevor sie zum Default wird.

1. Forschungsfragen-Übersicht

Forschungsfragen (Claude Code)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-prompt-correctness	Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?	aktiv	24	20/24 (83 %)	124
1.2	RQ-prompt-known-kata	Beeinflusst der Prompt-Stil (prose/user-story/example-mapping) bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?	aktiv	9	9/9 (100 %)	45
2.1	RQ-model-quality	Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?	aktiv	8	8/8 (100 %)	31
2.2	RQ-model-novel	Wie unterscheiden sich Opus 4.8, Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten, die stärker differenziert als die trainingsbekannte game-of-life?	aktiv	3	3/3 (100 %)	20

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
3.1	RQ-workflow-model	Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?	aktiv	6	6/6 (100 %)	49
4.1	RQ-tdd-quality	Wie wirkt sich die Workflow-Struktur (von oneshot ueber iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualitaet aus, und macht die TDD-Striktheit einen Unterschied?	aktiv	16	16/16 (100 %)	103
4.2	RQ-tdd-correctness	Unterscheidet sich die externe Korrektheit (verification_pct) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?	aktiv	7	7/7 (100 %)	36
4.3	RQ-context	Welche Form der Kontext-Strukturierung — isolierte Subagent-Kontexte pro TDD-Phase (v4.1), ein geteilter, akkumulierter Single-Context (v5.1), ein Hybrid mit Skill-basiertem Red/Green im Shared-Context und isoliertem Refactor-Subagent (v6.1) oder ein Hybrid mit isolierten Green- und Refactor-Subagents bei Shared-Context-Test-Liste/Red (v7.1) — fuehrt zu besserer Code-Qualitaet?	aktiv	4	4/4 (100 %)	21
4.4	RQ-pocock-vs-v62	Wie schneidet der externe Matt-Pocock-TDD-Skill (v9-pocock-tdd: Single-Skill, Inline-Phasen, Tail-Refactor) auf claim-office-example-mapping gegen die interne Default-Baseline v6.2-with-why-cleaned (Multi-Command + Refactor-Subagent, Per-Cycle-Refactor) ab — auf Korrektheit, Code-Qualitaet, TDD-Disziplin und Kosten?	aktiv	2	2/2 (100 %)	11
5.1	RQ-stability	Wie stabil sind Code-Qualitaet und TDD-Disziplin pro Workflow ueber Replikate, und unter welchen Bedingungen ist n=3 als Replikat-Anzahl ausreichend?	aktiv	6	5/6 (83 %)	59

Forschungsfragen (OpenCode)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-model-quality-oc	Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle (Opus 4.7 via Portkey + vier Nicht-Anthropic-Modelle aus dem Portkey-Catalog) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?	aktiv	6	6/6 (100 %)	30
1.2	RQ-model-novel-oc	Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?	aktiv	8	8/8 (100 %)	40

Forschungsfragen (Harness-übergreifend)

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-harness	Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität und TDD-Disziplin aus, wenn Modell, Workflow-Intention und Prompt-Stil konstant gehalten werden?	aktiv	6	6/6 (100 %)	38

Workflow-Entwicklung

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.1	RQ-pep-v6.1	Liefern psychologische Begründungen ('pep talks') in den Red- und Green-Skill-Prompts auf v6.1-Basis einen messbaren Code-Qualitäts- oder TDD-Disziplin-Vorteil ueber rein operationale Anweisungen?	aktiv	2	2/2 (100 %)	10
1.2	RQ-emoji-v6.1	Haben Decoration-Emojis (✅❌🔴🟢🔄📄🔔) in den Workflow-Prompts (Skills + Refactor-Agent + rules/tdd.md) auf v6.1-Basis einen messbaren Effekt auf Code-Qualität oder TDD-Disziplin?	aktiv	2	2/2 (100 %)	10
1.3	RQ-pep-emoji-v6.1	Sind die Effekte der pep- und emoji-Reduktionen auf v6.1-Basis additiv (zwei unabhängige Kanäle) oder gemeinsam getragen (ein 'Prompt-Drumherum'-Mechanismus)?	aktiv	4	4/4 (100 %)	25

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.4	RQ-pep-emoji-claim-office	Haeft der Interaktions-Befund aus RQ-pep-emoji-v6.1 (Pep+Emoji-Reduktion: Anti-Additivitaet bei refactorings_applied, Saettigung bei tests_passed_immediately, Korrektheit invariant) auch auf einer komplexeren Kata mit echten Mehrdeutigkeiten?	aktiv	4	4/4 (100 %)	22
1.5	RQ-why-block-effect-v6.1	Tragen Why-Blocke (kausale Begrueendungen neben MUSTs) auf v6.1-Basis einen messbaren TDD-Disziplin- oder Korrektheits-Vorteil ueber rein imperative Anweisungen — bei voll erhaltenem PEP?	aktiv	2	1/2 (50 %)	15
1.6	RQ-v62-cleanup-validation-v61-with-why	Veraendern die drei v6.5.1-Audit-Cleanups (Konsistenz, refactor.md-Entkopplung, tdd-experiment-mode-Reframing) — angewendet auf v6.1-with-why — messbar das Workflow-Verhalten auf claim-office, oder ist v6.2-with-why-cleaned eine verhaltens-aequivalente Hygiene-Variante der neuen Default-Baseline?	aktiv	2	2/2 (100 %)	16
1.7	RQ-v62-cleanup-validation-gol	Generalisiert das Cleanup-Aequivalenz-Ergebnis aus RQ-1.6 (claim-office) auch auf die trainings-bekannte game-of-life-Kata, oder zeigt v6.2-with-why-cleaned dort einen anderen Effekt als auf claim-office?	aktiv	2	2/2 (100 %)	15
1.8	RQ-audit-bundle-v62	Reproduziert das Audit-Bundle (Rationale-Ergaenzungen + Red-Phase-Hardening) auf der v6.2-with-why-cleaned-Basis die in der archivierten RQ-audit gegen v6.5-lean gemessenen Effekte (Disziplin-Plus, Streuungs-Schrumpf, Token/Wallclock-Aufschlag bei Korrektheits-Erhalt)?	aktiv	2	2/2 (100 %)	20
1.9	RQ-audit-bundle-claim-office	Generalisiert der RQ-1.8-Befund (Audit-Bundle wirkt Disziplin-stabilisierend und Code-Qualitaets-neutral auf v6.2-Basis × game-of-life) auch auf die novel claim-office-Kata, oder kippt das Pattern dort wie schon in RQ-1.4 fuer Reduktionen geschehen?	aktiv	2	2/2 (100 %)	16
1.10	RQ-refactor-vocab-v62	Verbessert ein additiver Vokabular-Block im refactor-Agent (Cyclomatic + Cognitive Complexity, Single Responsibility, Smell → Move-Tabelle) die Code-Qualitaet auf v6.2-with-why-cleaned-Basis, ohne Korrektheit oder Kosten signifikant zu beeintraechtigen?	aktiv	4	4/4 (100 %)	28

Kap.	RQ	Frage	Status	Cells	Coverage	n Runs
1.11	RQ-metric-driven-refactor-v62	Verbessert ein Refactor-Agent, der deterministische Metriken (ESLint smells, SonarJS cognitive complexity, McCabe cyclomatic complexity) selbst pre/post misst und APP-Mass parallel ausweist, die Code-Qualitaet auf claim-office gegenueber dem Baseline-v6.2-with-why-cleaned-Workflow — ohne Korrektheit oder TDD-Disziplin zu beschaedigen?	aktiv	2	2/2 (100 %)	13
1.12	RQ-end-refactor-v62	Verbessert ein metric-driven Refactor-Pass die Code-Qualitaet gegenueber dem Per-Cycle-Baseline-Workflow (v6.2-with-why-cleaned) — und greift der Hebel als rein per-cycle (v6.4-metric-driven-refactor) oder als zusaetzlicher Whole-src-End-Pass (v6.5-end-refactor) — ohne Korrektheit oder TDD-Disziplin zu beschaedigen, und haelt der Befund ueber zwei Kata-Typen (mehnteilige CLI-Codebasis claim-office vs einteilige Library game-of-life)?	aktiv	6	6/6 (100 %)	43
1.13	RQ-end-refactor-opus48	Haelt der v6.5-end-refactor-Befund aus RQ-1.12 (Korrektheit intakt, Code-Qualitaet >= v6.2, Token-Kosten ~v6.2) auf Opus 4.8 (no-thinking) — oder taeuscht der zusaetzliche End-Refactor-Pass auf dem neuen Modell die claim-office-Vollstaendigkeit aus (Bundle-Bruch-Muster aus RQ-1.9/RQ-1.10)?	aktiv	6	6/6 (100 %)	30

2. Experiment-Design

2.1 Variablen

Workflow — sechs Generationen (Details: [research/workflow-dev/workflow-construction.md](#) — Inventar):

Workflow	Aufbau	TDD-Streng
v1-oneshot	"Implementiere X."	keine
v2-iterative	"Plane Schritt für Schritt, dann implementiere."	keine
v3-basic-tdd	Inline TDD, kein Skill/Subagent (Self-Reporting)	minimal
v4-exact-subagents	Eigener Subagent pro Phase (Predictor + Red/Green/Refactor), fresh context	strikt, multi-context
v4.1-testlist-scope-fix	v4 mit Test-List-Scope-Patch	strikt, multi-context
v5-exact-single-context	Alle Phasen in einer Konversation, gleiches Phasen-Skript	strikt, single-context
v5.1-testlist-scope-fix	v5 mit Test-List-Scope-Patch (an v4.1 angeglichen)	strikt, single-context
v6-hybrid	Hybrid: inline TDD + nur Refactor als Subagent	strikt, hybrid
v6.1-hybrid-testlist-scope-fix	v6-hybrid mit Test-List-Scope-Patch (aktuelle Default-Basis)	strikt, hybrid
v6.1-no-pep	v6.1 ohne Pep-Talks (RQ-pep-Replikation)	strikt, hybrid
v7-hybrid-green-refactor	Wie v6, aber green und refactor als Subagent	strikt, mehr Isolation
v7.1-hybrid-green-refactor-testlist-scope-fix	v7 mit Test-List-Scope-Patch	strikt, mehr Isolation

Workflow	Aufbau	TDD-Strenges
v8a-delayed-refactor-agent	Oneshot → nachträgliche Tests → einmaliger End-Refactor-Agent (refactor.md aus v6.5.4)	delayed-refactor
v8b-delayed-refactor-native	Wie v8a, aber nativer Inline-Refactor im v3-Stil, kein Agent	delayed-refactor

Konfiguration: experiments/workflows/<variant>/ .claude/agents/ und .claude/rules/. Archivierte Varianten (v5.1-minimized, v6.2–v6.6, v6.5.x-Audits) liegen unter experiments/workflows/_archive/.

Workflow-Mechanik im Detail. Die sechs Generationen sind nicht nur eine Skala "mehr/weniger TDD", sondern eine systematische Variation der EXACT-Coding-Bausteine (Test-Liste, Red, Green, Refactor) und ihrer Kontext-Architektur:

- **v1-oneshot / v2-iterative — Vibe-Coding-Baselines (kein TDD).** Ein einzelner Agent liest die Anforderungen und schreibt Code in einem Schritt (v1) oder mit explizitem Plan/Checkliste (v2); Tests werden erst nachträglich auf Basis des Example Mappings hinzugefügt. Dient als Messlatte für den Wert von TDD selbst (siehe experiments/workflows/v1-oneshot/.claude/rules/experiment-mode.md).
- **v3-basic-tdd — Minimal-TDD ohne Struktur.** Ein einziger Agent mit minimaler Anweisung "use TDD" — keine Phasen-Prompts, keine Subagents. Claude entscheidet selbst, wie es den TDD-Prozess strukturiert. Misst, wie weit eine reine Aufforderung trägt (v3-basic-tdd/.claude/rules/experiment-mode.md).
- **v4-exact-subagents / v4.1-testlist-scope-fix — Strict TDD, multi-context.** Jede TDD-Phase läuft als spezialisierter Subagent in **isoliertem Kontext** (Task(subagent_type: "red") etc.): test-list → red → green → refactor. Hypothese: isolierte Kontexte erzwingen Disziplin, können aber Zustand zwischen Phasen verlieren. v4.1 ergänzt im test-list-Subagent die Pflicht "Cover every spec example" — schließt den dominanten Failure-Mode auf novel Katas (unvollständige Test-Liste) auf Opus 4.7.
- **v5-exact-single-context / v5.1-testlist-scope-fix — Strict TDD, single-context.** Identisches Phasen-Skript wie v4, aber alle Phasen laufen im **gleichen Kontext** als Skill-Calls (Skill(skill: "red") etc.) statt als Subagents. Hypothese: shared context erhält den Zustand, kann aber zu Disziplin-Verlust führen. v5.1 spiegelt v4.1 mit dem identischen Test-List-Scope-Patch.
- **v6-hybrid / v6.1-hybrid-testlist-scope-fix — Hybrid mit isoliertem Refactor.** Red und Green laufen inline als Skills im Shared-Context (wie v5), Refactor läuft als isolierter Subagent (wie v4). Hypothese: kombiniert die Spec-Kohärenz des Single-Context mit der Disziplin-Schärfung der Subagent-Isolation am kritischsten Punkt (Refactor). v6.1 ist die aktuelle Default-Basis und Champion über mehrere RQs. v6.1-no-pep testet die Reduktion psychologischer Begründungen in Red/Green.
- **v7-hybrid-green-refactor / v7.1-....-testlist-scope-fix — Hybrid mit isoliertem Green + Refactor.** Zusätzlich zur Refactor-Isolation aus v6 läuft auch Green als isolierter Subagent. Test-Liste und Red bleiben im Shared-Context. Prüft, ob mehr Isolation gleich besser ist (Pareto-dominiert von v6 auf game-of-life: spart Tokens, verliert Qualität und Korrektheit).
- **v8a-delayed-refactor-agent / v8b-delayed-refactor-native — Delayed-Refactor-Kontrolle.** Drei sequentielle Phasen ohne TDD-Cycles: (1) Oneshot-Implementation, (2) nachträgliche Tests gegen prompt.md mit Coverage-Pflicht, (3) ein einmaliger End-Refactor. v8a nutzt den refactor.md-Subagent aus v6.5.4 (APP + Naming + Mandatory-Attempt), v8b einen nativen Inline-Refactor im v3-Stil ohne Agent. Dient als Kontroll-Achse für die Hypothese "periodisches TDD-Refactor schlägt End-Refactor nach Vibe-Coding".

Tiefere Mechanik-Diskussion, Inventar der aktiven v6.1-Reduktionslinie und tragende RQ-Befunde stehen in research/workflow-dev/workflow-construction.md. Welche Marker das Parsing der TDD-Metriken treibt, dokumentiert experiments/workflows/MARKERS.md. Die archivierte v6.5.x-Linie liegt in experiments/workflows/_archive/ und research/_archive/workflow-dev-v1/.

Modell × Thinking (Lab-Varianten-IDs aus MODEL_CONFIGS in experiments/docker/run-batch.sh):

Lab-Varianten-ID	API-ID	Thinking	Routing
opus-4-7	claude-opus-4-7	Adaptive	Direct
opus-4-7-no-thinking	claude-opus-4-7	aus	Direct
sonnet-4-6	claude-sonnet-4-6	Extended	Direct
sonnet-4-6-no-thinking	claude-sonnet-4-6	aus	Direct
haiku-4-5	claude-haiku-4-5-20251001	Extended	Direct
haiku-4-5-no-thinking	claude-haiku-4-5-20251001	aus	Direct
opus-4-7-portkey	@vertex-eu-global/anthropic.claude-opus-4-7	Adaptive	Portkey
opus-4-7-portkey-no-thinking	@vertex-eu-global/anthropic.claude-opus-4-7	aus	Portkey
opus-4-6-portkey	@vertex-ai/anthropic.claude-opus-4-6	Adaptive	Portkey
opus-4-6-portkey-no-thinking	@vertex-ai/anthropic.claude-opus-4-6	aus	Portkey
sonnet-4-6-portkey	@vertex-ai/anthropic.claude-sonnet-4-6	Extended	Portkey
sonnet-4-6-portkey-no-thinking	@vertex-ai/anthropic.claude-sonnet-4-6	aus	Portkey
haiku-4-5-portkey	@vertex-ai/anthropic.claude-haiku-4-5@20251001	Extended	Portkey
haiku-4-5-portkey-no-thinking	@vertex-ai/anthropic.claude-haiku-4-5@20251001	aus	Portkey

Direct- und Portkey-Routings desselben Modells sind getrennte Varianten und werden nur per expliziter controls.model: {any: [...]}-Klausel pro RQ als gemeinsame Zelle gewertet.

Kata × Prompt-Stil (aktive Katas in experiments/katas/):

Kata-Basis	Prompt-Stile	Verifikations-Suite	Hinweis
game-of-life	prose, example-mapping, user-story	nein	Code-Qualität, groß (~40 LoC), vitest-basiert
game-of-life-cli	prose, example-mapping, user-story	ja	CLI-Variante mit externer Akzeptanz-Suite
mars-rover	prose, example-mapping, user-story	nein	mittel (~30 LoC), vitest-basiert
claim-office	prose, example-mapping, user-story	ja	Korrektheit, novel Versicherungs-Domäne (HPSMV/MHPCO), 15 Szenarien
claim-office-lite	prose, example-mapping, user-story	ja	Reduzierte claim-office-Variante (10 Szenarien) für Code-Qualitäts-Research

Prompt-Stile: - **prose**: Beschreibung der Regeln in Prosa, keine Test-Beispiele. - **example-mapping**: Regel + 1–3 konkrete Input/Output-Beispiele pro Regel. - **user-story**: "Als X möchte ich Y, damit Z" — Beschreibung ohne Beispiele.

2.2 Workflow → Prompt-Mapping

Aus methodischer Symmetrie (siehe Top- README.md , Abschnitt 'Methodology constraints'):

Workflow	erlaubte Prompt-Stile	Begründung
v1, v2	nur prose	Test-Beispiele in example-mapping wären für Non-TDD-Workflows ein verstecktes Test-Geschenk → unfair gegenüber den TDD-Workflows.
v3, v4(.1), v5(.1), v6(.1), v7(.1), v8a/b	alle drei	Beispiele dienen als natürliche Test-Cases — für TDD-/Refactor-Workflows ist das das Idealbild der Aufgabe.

3. Methodik

Die Run-Pipeline (Container → Run-Dir → pnpm-Workspace → headless `claude --print → analyze-run.sh → aggregate-by-query.py`) ist gegenüber dem vorigen Snapshot inhaltlich unverändert; die einzige Drift ist der Container-Pin der Claude-Code-CLI. Geprüft gegen `experiments/docker/Dockerfile`, `experiments/analyze-run.sh` und `experiments/aggregate-by-query.py` am 2026-05-31.

3.1 Run-Pipeline

- 1. Container-Image `docker-batch` (Node 22 slim, claude-code 2.1.156 gepinnt — Bump von 2.1.107 für Opus-4.8-Support) wird gestartet.
- 2. Run-Dir `runs/<timestamp>_<kata>_<workflow>_<model>/` wird angelegt; Workflow-Konfig (`.claude/agents/`, `.claude/rules/`) und Kata-Prompt (`prompt.md`) hinein kopiert.
- 3. pnpm-Workspace mit TypeScript, Vitest, ESLint+SonarJS aufgesetzt.
- 4. `claude --print "$(< prompt.md)"` läuft headless, ohne HITL.
- 5. `analyze-run.sh` schreibt `metrics.json` und `analysis-report.md`.
- 6. `aggregate-by-query.py <RQ>/` baut `runs.csv` und `summary.md` pro RQ.

3.2 Erfasste Metriken

Verbindliche Termini (Spalte "Term") sind im Top- README.md definiert — alternative Synonyme sind verboten, weil sie kollidieren oder mehrdeutig sind. Volle Metrik-Tabelle inklusive externer Referenzen (Stryker, SonarJS, McCabe-Paper etc.) im README Abschnitt "Metrics".

Korrektheit

Metrik	Term	Was misst es	Richtung
tests_passing	Korrektheit (innen)	Boolean: laufen die vom Agenten geschriebenen Vitest-Tests am Ende des Runs grün?	true = besser
verification_pct	Korrektheit (außen)	Anteil bestandener Verifikations-Szenarien aus einer externen Acceptance-Suite, die der Agent nie zu sehen bekommt (0.0–1.0). Nur für CLI-Katas mit <code><basename>-verification/</code> -Verzeichnis.	höher = besser

Effizienz

Metrik	Term	Was misst es	Richtung
duration_seconds	—	Wallclock-Sekunden des <code>claude --print</code> -Runs inkl. aller Subagent-Spawns	kleiner = besser
total_tokens	—	Summe aller Tokens (Input + Output + Cache) über alle Subagent-Spawns hinweg	kleiner = besser
context_utilization_pct	—	Finale Context-Window-Auslastung im Main-Context, in Prozent	informativ

Code-Mass & Umfang

Metrik	Term	Was misst es	Richtung
code_mass	Code-Mass (APP)	Gewichtete Summe der Produktiv-Code-Konstrukte (Konstanten, Invocations, Conditionals, Loops, Assignments — gestaffelte Gewichte nach Komplexität) gemäß <i>Absolute Priority Premise</i> (Micah Martin). Vergleicht Implementationen objektiver als reine LoC.	kleiner = besser
cc_loc	Produktiv-LoC	Produktiv-LoC ohne Tests, aus dem Clean-Code-Reporter	kleiner = besser (bei gleicher Korrektheit)
test_lines	Test-LoC	Anzahl Zeilen Test-Code (Vitest)	informativ
tests_total	—	Anzahl vom Agenten geschriebener Tests	informativ

Code-Qualität (ESLint + SonarJS)

Metrik	Term	Was misst es	Richtung
cc_longest_function	Spitzen-Komplexität	Längste Funktion in Zeilen — Proxy für die schlechteste Stelle im Code	kleiner = besser
cc_avg_loc_per_function	—	Mittlere Funktionsgröße in Zeilen	kleiner = besser
cc_median_loc_per_function	—	Median-Funktionsgröße (robust gegen einzelne lange Outlier)	kleiner = besser
cc_functions	—	Anzahl Funktionen	informativ
mccabe_max / mccabe_avg / mccabe_high_count	—	McCabe Cyclomatic Complexity pro Funktion: Maximum, Mittel, Anzahl über Schwellwert. Klassische Verzweigungs-Metrik.	kleiner = besser
cognitive_max / cognitive_avg / cognitive_high_count	—	SonarSource Cognitive Complexity pro Funktion: gewichtet Nesting und Control-Flow-Breaks stärker als McCabe, näher an menschlich wahrgenommener Komplexität. Diagnostisch tragende Hauptmetrik dieser Studie.	kleiner = besser
smell_total	Smell-Summe	Aggregierte Anzahl ESLint+SonarJS-Verstöße über alle Regeln	kleiner = besser
smell_complexity	—	Subset von <code>smell_total</code> : cognitive-complexity, max-depth, max-lines-per-function, max-params, no-nested-switch	kleiner = besser
smell_magic_numbers	—	Subset: ESLint <code>no-magic-numbers</code> - Verstöße	kleiner = besser
smell_duplication	—	Subset: SonarJS <code>no-duplicate-string</code> und verwandte Duplikations-Regeln	kleiner = besser
smell_code_quality	—	Subset: SonarJS <code>no-collapsible-if</code> , <code>no-redundant-jump</code> etc., plus ESLint <code>no-unreachable</code>	kleiner = besser
coverage_statements_pct	—	Statement-Coverage der vom Agenten geschriebenen Tests (in %)	höher = besser
coverage_branches_pct	—	Branch-Coverage der vom Agenten geschriebenen Tests (in %)	höher = besser

Test-Stärke

Metrik	Term	Was misst es	Richtung
mutation_score	Mutation-Score	Anteil der Stryker-Mutanten, die von der Test-Suite des Agenten gekillt werden (0.0–1.0): $(\text{Killed} + \text{Timeout}) / (\text{Killed} + \text{Survived} + \text{Timeout} + \text{NoCoverage})$. Hidden Metric — kommt in keinem Workflow-Prompt vor, daher Goodhart-resistent. Opt-in per RQ, nur für <code>tests_passing = true</code> .	höher = besser

TDD-Disziplin (aus `transcript.jsonl` + `transcript-subagents/`; getrieben von vier Markern in `experiments/workflows/MARKERS.md` — fehlt ein Marker, fällt die zugehörige Metrik still auf null)

Metrik	Term	Was misst es	Richtung
cycle_count	—	Anzahl Red-Green-Refactor-Zyklen pro Run	informativ (höher = feiner zerlegt)
refactorings_applied	—	Anzahl explizit angewandter Refactoring-Schritte	höher = besser (bei TDD-Workflows)
predictions_correct / predictions_total	—	Red-Phase-Vorhersagen über Compile-/Runtime-Failure: korrekt vs. gesamt. Tiefe des Code-Verständnisses des Agenten. Pro Cycle 1–2 Predictions je nach Workflow.	Quote höher = besser

Metrik	Term	Was misst es	Richtung
tests_passed_immediately	—	Anzahl Tests, die in der Red-Phase bereits grün waren — Indikator für Over-Implementation in vorherigen Green-Phasen	kleiner = besser
avg_red_seconds / avg_green_seconds / avg_refactor_seconds	—	Mittlere Phasendauer pro Cycle	informativ

3.3 Bewertungsgrundsätze

- **Korrektheit zuerst:** ein Run mit `tests_passing=false` zählt nicht als gleichwertige Lösung.
- **Pro Kata aggregieren:** Workflow×Modell-Tabellen werden ausschließlich pro Kata gebildet.
- **Effekt-Schwelle:** Bei $n=1$ pro Zelle gelten nur Differenzen mit Faktor ≥ 2 oder klar getrennten σ -Bändern als belastbar.

4. Ergebnisse

Forschungsfragen (Claude Code)

1.1 RQ-prompt-correctness — Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?

Datenbasis: 124 Runs · Coverage: 20/24 Zellen (83 %) bei $\min_replicates=5$.

Übersicht: Korrektheit (außen) nach Modell × Prompt-Stil × Thinking

Modell	Modus	prose	example-mapping	user-story
opus-4-7	-thinking	—	1.00 🏆 (n=3)	—
opus-4-6	-thinking	0.23	0.87 🏆	0.23
opus-4-6	+thinking	0.15	0.77 🏆	0.25
sonnet-4-6	-thinking	0.23	0.71 🏆	0.17
sonnet-4-6	+thinking	0.21	0.35 🏆	0.19
haiku-4-5	-thinking	0.00	0.00	0.00
haiku-4-5	+thinking	0.00	0.00	0.01

Werte: `mean(verification_pct)`, je $n=5$ (Opus 4.7: $n=3$, vorläufig). Höher = besser; 🏆 = bester Stil pro Zeile (Haiku-Zeilen: kein Effekt, alle Werte $\sim 0 \rightarrow$ kein Sieger).

Befunde:

- **F-prompt-correctness.1** — Schwache Modelle scheitern unabhängig vom Prompt-Stil
- **F-prompt-correctness.2** — Example-Mapping hebt Korrektheit massiv
- **F-prompt-correctness.3** — Thinking schadet bei Example-Mapping (Sonnet > Opus)
- **F-prompt-correctness.4** — User-Story $\approx$ Prose, keine messbare Wirkung auf Korrektheit
- **F-prompt-correctness.5** — Streuung bei Example-Mapping ist modellabhängig

Der dominante Befund: Example-Mapping hebt die Korrektheit (außen) auf der novel claim-office-Kata massiv — bei Opus 4.6 (no-thinking) von 0.23 (Prose) auf 0.87, ein Sprung von +64 pp; User-Story bleibt mit $\Delta \leq 6$ pp praktisch wirkungslos. Der Effekt ist modellabhängig: Haiku scheitert stilunabhängig bei 0.00, und Thinking schadet Example-Mapping (Sonnet -36 pp), weil das Modell die Beispiel-Semantik hinterfragt und Alternativ-Lesarten konstruiert. Caveat: nur eine novel Kata, die Opus-4.7-Zelle ist mit $n=3$ vorläufig. Details: research/questions-claude/1.1-prompt-style-correctness/findings.md.

1.2 RQ-prompt-known-kata — Beeinflusst der Prompt-Stil (prose/user-story/example-mapping) bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?

Datenbasis: 45 Runs · Coverage: 9/9 Zellen (100 %) bei $\min_replicates=5$.

Übersicht: verification_pct nach Prompt-Stil × Modell

Modell	prose	user-story	example-mapping
opus-4-6-portkey-no-thinking	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)
sonnet-4-6-portkey-no-thinking	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)	1.00 🏆 ($\sigma=0$)
haiku-4-5-portkey-no-thinking	0.24 ($\sigma=0.43$)	0.00 ($\sigma=0$)	0.63 🏆 ($\sigma=0.51$)

Höher = besser; 🏆 = bester Stil pro Zeile (Opus/Sonnet: alle drei Stile bei 1.00 gleichauf $\rightarrow$ Ties, alle 🏆).

Befunde:

- **F-prompt-known-kata.1** — Opus und Sonnet liefern stilunabhängig perfekte Korrektheit
- **F-prompt-known-kata.2** — Haiku scheitert kapazitätsbedingt, nicht stilbedingt
- **F-prompt-known-kata.3** — H1 bestätigt: Prompt-Stil differenziert bei starken Modellen nicht

- **F-prompt-known-kata.4** — H4 bestätigt: Mehrdeutigkeits-Mechanismus greift nicht bei trainingsbekannter Kata
- **F-prompt-known-kata.5** — H2 kann nicht bewertet werden: Code-Qualität nur bei funktionierenden Runs vergleichbar
- **F-prompt-known-kata.6** — RQ-prompt-correctness-Prognose bestätigt: Prompt-Stil differenziert nicht auf trainingsbekannter Kata
- **F-prompt-known-kata.7** — Verification-Adapter eliminiert Interface-Artefakte

Auf der trainingsbekannten game-of-life-Kata kompensieren starke Modelle jeden Stil-Unterschied vollständig: Opus 4.6 und Sonnet 4.6 (no-thinking) erreichen über alle drei Stile `verification_pct = 1.00` (30/30 Runs, $\sigma=0$). Haiku scheitert kapazitäts-, nicht stilbedingt — es zeigt ein bimodales Muster (Sofort-Aufgeber vs. Durchläufer), wobei Example-Mapping als Aktivierungsanker wirkt (0.63 vs. 0.00 bei User-Story). Damit ist bestätigt: Prompt-Stil differenziert nur dort, wo eine Kata echte Mehrdeutigkeiten trägt — auf trainingsbekanntem Code ist er als Control fixierbar. Details: research/questions-claude/1.2-prompt-style-known-kata/findings.md.

2.1 RQ-model-quality — Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?

Datenbasis: 31 Runs · Coverage: 8/8 Zellen (100 %) bei `min_replicates=3`.

Übersicht: Code-Qualität nach Modell (Mittelwerte)

Modell	code_mass	smell_total	mccabe_max	cognitive_max	cc_longest_function	verification_pct	n
opus-4-8	145.33 🏆	2.67	4.33	5.33	4.33 🏆	1.00 🏆	3
opus-4-8-no-thinking	186.33	3.00	4.00	4.00	11.67	1.00 🏆	3
opus-4-7	159.00	2.33 🏆	3.33 🏆	3.00 🏆	7.00	1.00 🏆	3
opus-4-7-no-thinking	166.60	2.60	4.50	4.40	8.10	1.00 🏆	10
opus-4-6-portkey	173.00	4.33	6.67	12.00	19.33	1.00 🏆	3
opus-4-6-portkey-no-thinking	175.67	4.33	7.67	13.00	18.67	1.00 🏆	3
sonnet-4-6	178.00	5.67	6.33	11.00	21.67	1.00 🏆	3
sonnet-4-6-no-thinking	166.67	3.33	6.00	5.00	15.00	0.73	3

Bester Wert pro Spalte fett + 🏆. Kleiner = besser (außer `verification_pct`: größer = besser). Quality-Trophies sind korrektkeits-geged (sonnet-4-6-no-thinking mit 0.73 ausgenommen); `verification_pct`: sieben Zellen gleichauf bei 1.00 → Ties, alle 🏆.

Befunde:

- **F-model-quality.1** — Korrektheit (innen + außen) auf v4 ist nahezu modellunabhängig perfekt
- **F-model-quality.2** — Modell-Ranking: Opus-4.8 und Opus-4.7 an der Spitze, deutlich vor Opus-4.6; Sonnet im Mittelfeld
- **F-model-quality.3** — Thinking wirkt nicht uniform; bei Opus 4.8 stark auf Code-Größe, bei Opus 4.6 neutral, bei Sonnet negativ auf `cognitive_max`
- **F-model-quality.4** — Token-Kosten: Opus 4.8 ist der teuerste, Sonnet/Opus 4.7 die günstigsten; Wallclock einheitlich
- **F-model-quality.5** — Vertrags-Konformität unter explizitem API-Vertrag fast vollständig erreicht; ein Sonnet-Ausreißer redefiniert `Cell` als Objekt

Bei phasen-isoliertem strikten TDD ist Korrektheit nahezu modellunabhängig perfekt (7/8 Zellen `verification_pct = 1.00`), sodass Code-Qualität differenziert. Die jüngsten Opus-Generationen führen klar: Opus 4.8 liefert den kompaktesten Code überhaupt (`code_mass` 145, `cc_longest_function` 4.3 — neue Bestwerte), Opus 4.7 die niedrigsten reinen Komplexitäts-Scores; beide deutlich vor Opus 4.6 (`cognitive_max` -Faktor ~4×). Der Preis von Opus 4.8 ist das höchste Token-Budget (~3.8 M, ~1.5× Opus 4.7). Thinking wirkt nicht uniform — bei Sonnet verschlechtert es `cognitive_max`. Caveat: nur eine Kata, ein Workflow, n=3 pro Zelle. Details: research/questions-claude/2.1-model-effect-code-quality/findings.md.

2.2 RQ-model-novel — Wie unterscheiden sich Opus 4.8, Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten, die stärker differenziert als die trainingsbekannte game-of-life?

Datenbasis: 20 Runs · Coverage: 3/3 Zellen (100 %) bei `min_replicates=5`.

Übersicht (Primär `verification_pct` höher = besser; Code-Qualität/Kosten kleiner = besser)

Modell	n	verification_pct ↑	σ	cognitive_max ↓	mccabe_max ↓	smell_total ↓	total_tokens ↓	duration_s ↓
opus-4-8-no-thinking	5	0.92 🏆	0.09	7.4 🏆	7.0 🏆	1.2 🏆	31.0 M	5264
opus-4-7-no-thinking	10	0.67	0.36	10.5	7.9	1.8	13.7 M 🏆	3693 🏆
opus-4-6-portkey-no-thinking	5	0.93 🏆	0.08	22.2	10.6	5.6	15.1 M	4416

4-6 (0.93) und 4-8 (0.92) sind innerhalb 0.1 σ ununterscheidbar → beide 🏆; 4-7 fällt mit 0.67 (bimodal) ab. Qualitäts-Pokale gehen an opus-4-8 (hoch-korrekt → echte Vollimplementierungen); die Token-/Duration-Pokale für 4-7 sind wegen teils unvollständiger Specs mit Vorsicht zu lesen.

Befunde:

- **F-model-novel.1** — opus-4-8 und opus-4-6 lösen claim-office zuverlässig, opus-4-7 nicht
- **F-model-novel.2** — Workflow×Modell-Interaktion ist der dominierende Effekt

- **F-model-novel.3** — Korrektheit differenziert stärker als Code-Qualität
- **F-model-novel.4** — Präziserer Mechanismus auf opus-4-7: Test-Listen-Vollständigkeit, nicht Subagent-Isolation
- **F-model-novel.5** — opus-4-8 erkaufte beste Code-Qualität mit ~2× Token-Kosten

Auf der novel claim-office-Kata trennt die Korrektheit selbst die Modelle (auf game-of-life tat das nur die Code-Qualität): opus-4-8 (0.92) und opus-4-6 (0.93) lösen sie zuverlässig und eng gestreut, opus-4-7 nicht (0.67, bimodal mit Runs bis 0.20). Die naive Erwartung "neuer = besser" trägt nicht — beide Ränder schlagen das mittlere Modell. opus-4-8 vereint die Spec-Treue von 4-6 mit der besten Code-Qualität (cognitive_max 7.4 vs. 22.2), erkaufte das aber mit ~2× Token-Kosten. Caveat: n=5, opus-4-6 läuft über Portkey (Routing-Confound), opus-4-8 nur auf v4 erhoben. Details: [research/questions-claude/2.2-model-effect-novel-kata/findings.md](#).

3.1 RQ-workflow-model — Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?

Datenbasis: 49 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Übersicht: **verification_pct** (claim-office-example-mapping) nach Workflow × Modell

Workflow	opus-4-7 (n)	opus-4-6 (n)
v4-exact-subagents	0.67 (10)	0.93 (5) 🏆
v5-exact-single-context	0.87 (10)	0.87 (5)
v6-hybrid	1.00 (5) 🏆	0.68 (15)

Höher = besser; 🏆 je Modell-Spalte — die Kernaussage ist gerade, dass der Sieger modell-abhängig wechselt.

Befunde:

- **F-workflow-model.1** — v4 und v6 tauschen je nach Modell die Plätze
- **F-workflow-model.2** — Mechanismus: Orchestrierungs-Delegation vs. expliziter Subagent-Prompt

Es gibt keinen universell besten Workflow: v4 (phasen-isolierte Subagents) und v6 (Hybrid mit isoliertem Refactor) tauschen je nach Modell die Plätze. v6 ist Opus-4.7-Optimum (1.00), aber auf Opus 4.6 instabil (0.68); v4 ist auf Opus 4.6 stabil (0.93), auf Opus 4.7 bimodal (0.67); v5 bleibt modell-unabhängig konstant (0.87). Mechanismus: v6 delegiert die Orchestrierung ans Modell (Skill-Invocation im Shared-Context) — das beherrscht 4.7, während 4.6 in ~40 % der Runs die Claim-Hälfte der Spec verliert. Caveat: nur claim-office, opus-4-8 auf dieser Achse noch nicht erhoben. Details: [research/questions-claude/3.1-workflow-model-interaction/findings.md](#).

4.1 RQ-tdd-quality — Wie wirkt sich die Workflow-Struktur (von oneshot ueber iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualität aus, und macht die TDD-Striktheit einen Unterschied?

Datenbasis: 103 Runs · Coverage: 16/16 Zellen (100 %) bei min_replicates=5.

Übersicht — **Code-Qualität pro Workflow** (kleiner = besser; 🏆 = bester Wert pro Spalte; nie über Katas gemittelt).

Kata: game-of-life

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v1-oneshot	10	18.8	12.8	31.7	4.8	33.6	155.0
v2-iterative	10	16.2	11.6	32.1	4.1	32.5	157.8
v3-basic-tdd	10	21.8	13.7	32.5	6.0	31.9	165.6
v4.1-testlist-scope-fix	5	6.4 🏆	5.0 🏆	16.4	2.4 🏆	32.0	156.6
v5.1-testlist-scope-fix	5	17.6	10.2	20.8	4.8	26.6 🏆	154.0
v6.1-hybrid-...	10	6.5	5.2	14.2 🏆	2.4 🏆	29.2	153.7
v8a-delayed-refactor-agent	5	10.6	7.4	17.6	3.0	31.2	142.0 🏆
v8b-delayed-refactor-native	5	9.0	6.8	17.6	2.4 🏆	31.0	145.8

Kata: claim-office

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v1-oneshot	5	12.2	8.4	40.4	11.6	269.4	835.4
v2-iterative	5	11.4	8.4	41.4	15.8	268.6	851.0
v3-basic-tdd	5	19.8	15.4	51.6	16.8	317.4	992.4
v4.1-testlist-scope-fix	5	26.8 🚩	16.0 🚩	40.8	13.2	156.8 🏆	621.6 🏆
v5.1-testlist-scope-fix	6	14.8	10.2	32.7	6.8	167.2	692.7
v6.1-hybrid-...	7	5.7 🏆	5.7 🏆	18.1 🏆	1.3 🏆	191.1	861.3
v8a-delayed-refactor-agent	5	7.4	6.6	28.4	4.0	245.6	813.8

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	cc_loc	code_mass
v8b-delayed-refactor-native	5	11.0	8.0	35.8	6.2	238.8	780.2

⚠ v4.1-claim-office ist bimodal (cognitive_max $\sigma=24$, max=68). Korrektheit: auf game-of-life alle 8 Workflows verification_pct = 1.00; auf claim-office 0.28 (v1/v2 Vibe) bis 1.00.

Befunde:

- **F-tdd-quality.1** — Strikte phasen-strukturierte Workflows mit Refactor-Phase senken die Komplexitäts-Spitzen drastisch
- **F-tdd-quality.2** — Naives „use TDD“ (v3) bringt auf game-of-life keinen Komplexitäts-Vorteil gegenüber Non-TDD (v1/v2)
- **F-tdd-quality.3** — Single-Context (v5.1) verliert den Komplexitäts-Vorteil der phasen-isolierten Subagents (v4.1) — aber nur auf game-of-life
- **F-tdd-quality.4** — Korrektheit ist workflow-abhängig auf novel Kata; v1/v2 Vibe-Coding kollabiert auf claim-office
- **F-tdd-quality.5** — Kostenspanne zwischen Workflows ist eine Größenordnung; strikte Workflows sind 5–50× teurer; Kata-Komplexität skaliert linear
- **F-tdd-quality.6** — Vibe + End-Refactoring erreicht Volumen-Niveau der strikten TDD-Workflows zu Non-TDD-Kosten; Verzweigungs-Komplexität bleibt schwächer
- **F-tdd-quality.7** — Subagent-Mechanismus für End-Refactor schlägt Slash-Command auf großer Kata; gleichauf auf kleiner Kata
- **F-tdd-quality.8** — Test-Schreib-Phase rettet Korrektheit auf novel Kata; reines Vibe-Coding scheitert
- **F-tdd-quality.9** — v6.1-Hybrid ist der robusteste TDD-Workflow über beide Katas; v4.1 ist kata-instabil

Strikte phasen-strukturierte Workflows mit Refactor-Phase senken die Komplexitäts-Spitzen drastisch (cognitive_max ~6 vs. ~16–22 bei Non-TDD und naivem „use TDD“). Der Hybrid mit isoliertem Refactor-Subagent ist der einzige Workflow, der auf beiden Katas in den Top-2 landet; die voll phasen-isolierte Variante bricht auf claim-office bimodal ein (cognitive_max 26.8, max 68). Der naive „use TDD“-Ansatz (v3) ohne erzwungenen Red-Green-Refactor-Takt bringt keinen Vorteil — auf claim-office produziert er sogar den schwersten Code der Matrix. Der Qualitäts-Hebel ist die strukturierte Refactor-Disziplin im Takt, nicht das TDD-Etikett. Diese Qualität kostet: 5–16× mehr Tokens als Vibe+End-Refactor bei gleicher Korrektheit; der Mehrwert ist Verzweigungs-Komplexität, nicht Korrektheit. Caveat: nur opus-4-7-no-thinking, n=5 pro claim-office-Zelle. Details: research/questions-claude/4.1-tdd-effect-code-quality/findings.md.

4.2 RQ-tdd-correctness — Unterscheidet sich die externe Korrektheit (verification_pct) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?

Datenbasis: 36 Runs · Coverage: 7/7 Zellen (100 %) bei min_replicates=3.

Übersicht — Korrektheit pro Workflow (höher = besser; 🏆 = bester Wert pro Spalte)

Workflow	n	verification_pct (mean ± std)	verification_passed / 15 (min – max)	tests_passing
v3-basic-tdd	5	1.00 ± 0 🏆	15 – 15	100 % 🏆
v4.1-testlist-scope-fix	5	0.96 ± 0.09	12 – 15	100 % 🏆
v5.1-testlist-scope-fix	6	1.00 ± 0 🏆	15 – 15	100 % 🏆
v6.1-hybrid-...	3	1.00 ± 0 🏆	15 – 15	100 % 🏆
v7.1-hybrid-green-refactor-...	3	0.98 ± 0.04	14 – 15	100 % 🏆

Befunde:

- **F-tdd-correctness.1** — Drei von fünf TDD-Workflows lösen claim-office perfekt; v4.1 und v7.1 verlieren vereinzelt Szenarien
- **F-tdd-correctness.2** — v4.1 erreicht Korrektheit nur über drastisch höheren Aufwand pro Zyklus
- **F-tdd-correctness.3** — Predictions-Rate-Vergleich ist verzerrt durch ungleiche Vorhersage-Basis
- **F-tdd-correctness.4** — Wallclock-Spanne ist 10×, Token-Spanne 9×; keine Korrektheits-Korrelation

Auf claim-office unter Opus 4.7 ist Korrektheit kein knappes Gut: drei der fünf TDD-Workflows (Minimal-TDD, Single-Context, Hybrid) lösen alle 15 Szenarien perfekt; die zwei mit isoliertem Green-Subagent tragen je einen Ausreißer. Bemerkenswert ist die Aufwands-Spreizung bei gleicher Korrektheit — die voll isolierte Variante fährt im Schnitt 44.6 Cycles (vs. 3.8 bei Minimal-TDD), die Wallclock-Spanne ist 10×, die Token-Spanne 9×, ohne jede Korrektheits-Korrelation. Der Wert struktureller Workflows liegt also nicht in der Korrektheit, sondern in der Code-Qualität (siehe RQ-context). Caveat: n=3 in mehreren Zellen. Details: research/questions-claude/4.2-tdd-effect-correctness/findings.md.

4.3 RQ-context — Welche Form der Kontext-Strukturierung — isolierte Subagent-Kontexte pro TDD-Phase (v4.1), ein geteilter, akkumulierter Single-Context (v5.1), ein Hybrid mit Skill-basiertem Red/Green im Shared-Context und isoliertem Refactor-Subagent (v6.1) oder ein Hybrid mit isolierten Green- und Refactor-Subagents bei Shared-Context-Test-Liste/Red (v7.1) — führt zu besserer Code-Qualität?

Datenbasis: 21 Runs · Coverage: 4/4 Zellen (100 %) bei min_replicates=3.

Übersicht — Code-Qualität, Korrektheit, Kosten (kleiner = besser außer verification_pct; 🏆 = bester Wert pro Spalte, bei Spread < 1 σ auf alle nahen Werte verteilt)

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	code_mass	cc_loc	verification_pct	duration_seconds	total_tokens
v4.1 (alle isoliert)	5	26.8 ± 24.1	16.0 ± 9.0	40.8 ± 27.1	13.2 ± 7.5	621.6 🏆	156.8 🏆	0.96	3 229	14.10 M 🏆
v5.1 (alle geteilt)	6	14.8 ± 4.2	10.2 ± 2.6	32.7 ± 10.2	6.8 ± 7.6	692.7	167.2	1.00 🏆	641 🏆	18.73 M

Workflow	n	cognitive_max	mccabe_max	cc_longest_function	smell_total	code_mass	cc_loc	verification_pct	duration_seconds	total_tokens
v6.1 (Refactor isoliert)	3	4.3 🏆	5.0 🏆	16.7 🏆	1.3 🏆	920.7	184.3	1.00 🏆	1 424	30.16 M
v7.1 (Green + Refactor isoliert)	3	5.0 🏆	4.67 🏆	19.3 🏆	2.3 🏆	801	187.3	0.98	1 970	26.11 M

tests_passing und completed_within_budget in allen vier Zellen 100 %.

Befunde:

- **F-context.1** — Refactor-Subagent liefert den Komplexitäts-Vorteil; zusätzliche Green-Isolation ändert das Bild nicht
- **F-context.2** — Refactor-Subagent verteilt Funktionalität auf mehr Bausteine; Green-Isolation bremst den Mehr-Code-Effekt
- **F-context.3** — Korrektheit unterscheidet die Architekturen nicht
- **F-context.4** — Vier sehr unterschiedliche Kosten-Profile
- **F-context.5** — Zwei Hybrid-Positionen mit ähnlicher Code-Qualität, unterschiedlichem Kosten-Profil

Der Komplexitäts-Vorteil entsteht ausschließlich aus dem **isolierten Refactor-Subagent** — dem gemeinsamen Element der beiden Hybrid-Architekturen, die praktisch gleichauf die niedrigsten Spitzen erreichen (cognitive_max 4.3 / 5.0). Zusätzliche Green-Isolation bringt keinen weiteren Hub; volle Phasen-Isolation schadet auf claim-office sogar (höchste Komplexität und Streuung, $\sigma_{\text{cognitive}}=24$). Korrektheit unterscheidet die Architekturen nicht (alle ~1.00). Die vier Kosten-Profile divergieren stark: Single-Context ist 5× schneller, voll-isoliert am token-sparsamsten, die Hybride am teuersten. Caveat: nur claim-office, n=3 in den Hybrid-Zellen; Cross-Kata-Replikation offen. Details: [research/questions-claude/4.3-tdd-context-engineering/findings.md](#).

4.4 RQ-pocock-vs-v62 — Wie schneidet der externe Matt-Pocock-TDD-Skill (v9-pocock-tdd: Single-Skill, Inline-Phasen, Tail-Refactor) auf claim-office-example-mapping gegen die interne Default-Baseline v6.2-with-why-cleaned (Multi-Command + Refactor-Subagent, Per-Cycle-Refactor) ab — auf Korrektheit, Code-Qualität, TDD-Disziplin und Kosten?

Datenbasis: 11 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=3.

Übersicht (v6.2-with-why-cleaned n=8 vs. v9-pocock-tdd n=3, claim-office-example-mapping)

Achse	v6.2-with-why-cleaned	v9-pocock-tdd	Sieger
verification_pct (höher = besser)	0.96 ± 0.09	1.00 ± 0 🏆	Pocock leicht
tests_passing rate	100 %	100 %	Tie 🏆🏆
cognitive_max (kleiner = besser)	5.00 ± 1.77 🏆	14.33 ± 1.53	v6.2
mccabe_max (kleiner = besser)	4.50 ± 0.76 🏆	11.67 ± 0.58	v6.2
cc_longest_function (kleiner = besser)	12.38 ± 1.41 🏆	32.33 ± 1.53	v6.2
smell_total (kleiner = besser)	0.38 ± 0.74 🏆	6.67 ± 8.96	v6.2
code_mass (kleiner = besser)	878.5 ± 91	748.3 ± 62 🏆	Pocock
duration_seconds (kleiner = besser)	2530 ± 401	570 ± 106 🏆	Pocock
total_tokens (kleiner = besser)	44.4 M ± 3.4 M	13.1 M ± 4.6 M 🏆	Pocock
refactorings_applied	24.88 ± 6.90	0 ± 0	by-design
cycle_count	37.38 ± 1.60	14.00 ± 3.46	by-design
tests_passed_immediately (kleiner = strikter)	15.12 ± 5.84	2.33 ± 4.04 🏆	Pocock
predictions_correct_rate (höher = besser)	97.2 % 🏆	89.9 %	v6.2

Befunde:

- **F-4.4.1** — Pocock und v6.2 gleichwertig korrekt
- **F-4.4.2** — v6.2 produziert sauberer Code, Pocock kompakter
- **F-4.4.3** — Pocock ~70–78 % günstiger
- **F-4.4.4** — Tail-Refactor löst auf claim-office nicht aus
- **F-4.4.5** — Pocock macht weniger, größere Schritte
- **F-4.4.6** — Pocock skippt seltener (strikterer Vertical-Slice)

Der externe Matt-Pocock-TDD-Skill (Single-Skill, Inline-Phasen, Tail-Refactor) ist auf claim-office gleichwertig korrekt wie die interne Default-Baseline (beide ~100 %) und dabei ~70–78 % günstiger (570 s vs. 2530 s; 13 M vs. 44 M Tokens). Den Preis zahlt er bei der Code-Qualität: cognitive_max 14.3 vs. 5.0, cc_longest_function 32 vs. 12. Der Grund ist mechanistisch klar — der Tail-Refactor löst in 3/3 Runs nie aus (refactorings_applied = 0), während die Baseline per-Cycle refactoriert. Caveat: Pocock-Zelle nur n=3, Effektgrößen aber > 3 σ . Details: [research/questions-claude/4.4-external-tdd-pocock-vs-v62/findings.md](#).

5.1 RQ-stability — Wie stabil sind Code-Qualitaet und TDD-Disziplin pro Workflow ueber Replikate, und unter welchen Bedingungen ist n=3 als Replikat-Anzahl ausreichend?

Datenbasis: 59 Runs · Coverage: 5/6 Zellen (83 %) bei min_replicates=10.

Übersicht: Code-Qualität nach Workflow (n=10) (kleiner = besser; bester Wert pro Spalte fett)

Workflow	code_mass	smell_total	mccabe_max	cognitive_max	cc_longest_function	n
v1-oneshot (prose)	155.00	4.80	12.80	18.80	31.70	10
v2-iterative (prose)	157.80	4.10	11.60	16.20	32.10	10
v3-basic-tdd (EM)	165.60	6.00	13.70	21.80	32.50	10
v4-exact-subagents (EM)	166.60	2.60	4.50	4.40	8.10	10
v5-exact-single-context (EM)	152.60	4.10	8.90	14.50	17.40	10
v6-hybrid (EM)	158.60	2.20	4.50	5.20	13.10	10

Befunde:

- **F-stability.1** — RQ-tdd-quality-Hauptbefund (v4 dominiert Code-Komplexität, v3 ist Schlusslicht) repliziert bei n=10 mit gleichem Vorzeichen
- **F-stability.2** — Workflow-Stabilität ist nicht uniform; v4 hat 10 %-Outlier-Rate trotz tiefem typischen Wert; v5 ist breitestes Workflow
- **F-stability.3** — Bei n=3 ist die volle Workflow-Rangordnung nur in ~25–60 % der Fälle korrekt; v4 als "Bester" ist robuster
- **F-stability.4** — Korrektheit bleibt bei n=10 modell-/workflow-unabhängig 100 %
- **F-stability.5** — Token-Verbrauch zeigt extrem hohe Streuung bei v4 und v5
- **F-stability.6** — TDD-Disziplin bildet workflow-charakteristische Banden
- **F-stability.7** — Test-Stärke (mutation_score) hat eigenes Stabilitätsprofil; v6-hybrid ist stabiler Workflow, v4 instabiler

Bei n=10 repliziert der Hauptbefund vollständig: v4 dominiert die Komplexitäts-Metriken, v3 ist Schlusslicht — kein n=3-Artefakt. Die Stabilität ist aber nicht uniform: v4 ist im typischen Fall am stabilsten (IQR 1.25), trägt jedoch eine 10 %-Outlier-Rate (ein Run mit cognitive_max 17), v5 streut breit, der Hybrid (v6) ist kompakt ohne Tail (alle 10 Runs ∈ [1, 7], 0 % Outlier). Subsampling zeigt: n=3 rekonstruiert die volle Rangordnung nur in 25–60 % der Fälle — robuste Einzelaussagen ("v4 bester") ja, feine Mittelfeld-Ränge nein. Auf mutation_score ist v6 der stabilste Workflow. Caveat: nur game-of-life, nur opus-4-7-no-thinking. Details: [research/questions-claude/5.1-workflow-stability/findings.md](#).

Forschungsfragen (OpenCode)

1.1 RQ-model-quality-oc — Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle (Opus 4.7 via Portkey + vier Nicht-Anthropic-Modelle aus dem Portkey-Catalog) in Code-Qualität und TDD-Disziplin auf game-of-life-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?

Datenbasis: 30 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Übersicht (game-of-life-example-mapping; kleiner = besser außer wo notiert; Quality-/Cost-Trophies korrektheits-gegated bei verification_pct = 1.0)

Metrik	Richtung	opus-4-7-portkey	glm-5-1	gemin-3-5-flash	kimi-k2-6	deepseek-v4-flash	deepseek-v4-pro
verification_pct (mean)	höher	1.00 🏆	1.00 🏆	1.00 🏆	0.57	1.00 🏆	0.85
smell_total (mean)	kleiner	3.6	2.8 🏆	4.0	4.4	4.0	4.2
cognitive_max (mean)	kleiner	11.4 🏆	11.6 🏆	16.0	9.4	13.2	11.4
mccabe_max (mean)	kleiner	7.6	7.0 🏆	10.4	7.6	9.4	8.6
cc_longest_function (mean)	kleiner	18.6 🏆	19.8	18.6 🏆	15.2	27.6	15.0
lines_of_code (mean)	kleiner	38.2 🏆	46.4	52.2	22.4	44.8	24.6
duration_seconds (mean)	kleiner	231	835	153 🏆	1083	612	2381
total_tokens (mean)	kleiner	1.82 M 🏆	2.96 M	2.80 M	2.28 M	2.71 M	2.82 M
cost_usd (mean, \$/perfect-run)	kleiner	\$1.84	\$0.74	\$1.06	\$2.65	\$0.10 🏆	\$0.46

Kimi (0.57) und DeepSeek-V4-Pro (0.85) fallen aus dem Trophy-Pool (verification_pct < 1.0).

Befunde:

- **F-1.1** — Opus 4.7 schreibt die kompakteste Implementierung
- **F-1.2** — GLM 5.1 hält Opus-Niveau in Komplexität
- **F-1.3** — Kimi-K2 schreibt zu wenige Tests, scheitert an externer Verifikation
- **F-1.4** — Gemini 3.5 Flash: schnell, aber komplexester Code
- **F-1.5** — Skill-Tool-Compliance ist modellabhängig
- **F-1.6** — DeepSeek-V4-Flash: günstigster Pfad zur korrekten Lösung
- **F-1.7** — DeepSeek-V4-Pro: Skill-Compliance-Champion, aber Tail-Risk in Duration

Über den OpenCode-Harness liefert Opus 4.7 die kompakteste Implementierung (38 LoC, Median 3.3 LoC/Funktion) bei voller Korrektheit und niedrigstem Token-Verbrauch. GLM 5.1 hält Opus-Niveau in der Komplexität (`cognitive_max` 11.6 vs. 11.4) zu OpenRouter-Preisen. Der günstigste Pfad zur korrekten Lösung ist DeepSeek-V4-Flash mit \$0.10/Run — eine Größenordnung unter Opus — bei 5/5 perfekter Korrektheit. Kimi scheitert an zu wenigen Tests (intern grün, extern 0.57); Gemini Flash ist am schnellsten, aber schreibt den komplexesten Code. Caveat: harness-/modell-erweitert (Nicht-Anthropic via OpenCode), n=5, eine Kata. Details: [research/questions-opencode/1.1-model-quality-oc/findings.md](#).

1.2 RQ-model-novel-oc — Wie unterscheiden sich fünf via OpenCode-Harness erreichbare Modelle in Korrektheit und TDD-Disziplin auf claim-office-example-mapping mit dem v5.1-testlist-scope-fix-oc-Workflow?

Datenbasis: 40 Runs · Coverage: 8/8 Zellen (100 %) bei min_replicates=5.

Übersicht (claim-office-example-mapping; verification_pct höher = besser, Rest kleiner = besser; Quality-/Cost-Trophies korrektkeits-gegated bei verification_pct = 1.0)

Metrik	Richtung	opus-4-7-portkey	glm-5-1	mistral-medium-3-5	kimi-k2-6	gemini-3-5-flash	deepseek-v4-flash	deepseek-v4-pro	minimax-m2-7
verification_pct (mean)	höher	1.00 🏆	1.00 🏆	0.95	0.84	0.80	0.60	0.60	0.04
smell_total (mean)	kleiner	0.8 🏆	4.0	23.6	20	18	13.4	16.6	10.2
cognitive_max (mean)	kleiner	9.8 🏆	12.2	74.8	21.8	40.2	11.6	17.4	11.4
mccabe_max (mean)	kleiner	7.6 🏆	9.2	33.6	17.6	23.4	9.2	11.0	7.6
code_mass (mean)	kleiner	759.6 🏆	816	712.6	741	526	566.2	554.6	364.4
cost_usd (mean, \$/Run)	kleiner	\$5.90	\$2.10 🏆	\$24.69 †	\$2.78	\$2.23	\$0.28	\$0.11	\$2.40
duration_seconds (mean)	kleiner	664 🏆	1726	4051	1811	395	1279	956	1428

Nur Opus 4.7 und GLM 5.1 erreichen vpct = 1.0 → Trophy-Pool. † Mistral-Cost ist ein OpenCode-Caching-Artefakt (fehlender `prompt_cache_key`), nicht Mistral-Pricing.

Befunde:

- **F-1.1** — Opus 4.7 und GLM 5.1 erreichen vollständige Korrektheit; Tradeoff Code-Qualität ↔ Kosten
- **F-1.2** — Kimi K2.6 und Gemini 3.5 Flash: Spitzen-Korrektheit mit Varianz-Tail
- **F-1.3** — MiniMax M2.7: stabiles Spec-Misverständnis, kein Einzelfall
- **F-1.4** — Predictions-Format-Compliance ist NICHT prädiktiv für Korrektheit
- **F-1.5** — Code-Mass-Spread innerhalb Modell: Flash und MiniMax bimodal/breit
- **F-1.6** — Cost-Effizienz pro perfektem Lauf: GLM 5.1 deterministisch UND günstig
- **F-1.7** — Mistral Medium 3.5: hohe Korrektheit gegen hohe Komplexität und höchste Kosten
- **F-1.8** — DeepSeek V4 (flash + pro): Workflow-Compat-Drop dominiert über Spec-Verstehen

Auf der novel claim-office-Kata erreichen nur Opus 4.7 und GLM 5.1 über alle fünf Replikate perfekte Korrektheit. GLM ist dabei 2.8× günstiger als Opus, zahlt aber mit etwas höherer Komplexität — ein klarer Qualität ↔ Kosten-Tradeoff. Die übrigen Modelle zeigen charakteristische Failure-Modi: MiniMax ein stabiles Spec-Misverständnis (0.04), DeepSeek-V4 einen bimodalen CLI-Vertrags-Drop (0.60), Mistral hohe Korrektheit gegen 6–8× höhere Komplexität. Prediction-Format-Compliance ist nicht prädiktiv für Korrektheit. Caveat: harness-/modell-erweitert, n=5, Mistral-Kosten integrations-verzerrt. Details: [research/questions-opencode/1.2-model-novel-kata-oc/findings.md](#).

Forschungsfragen (Harness-übergreifend)

1.1 RQ-harness — Wie wirkt sich der Harness-Wechsel (Claude Code vs OpenCode vs pi) auf Korrektheit, Code-Qualität und TDD-Disziplin aus, wenn Modell, Workflow-Intention und Prompt-Stil konstant gehalten werden?

Datenbasis: 38 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Übersicht (alle Zellen opus-4-7-portkey-no-thinking, example-mapping, gleicher Workflow je Harness; verification_pct / refactorings höher = besser, Rest kleiner = besser)

Outcome	CC × claim	OC × claim	pi × claim	CC × GOL	OC × GOL	pi × GOL
verification_pct	0.96	1.00 🏆	1.00 🏆	1.00 🏆	1.00 🏆	1.00 🏆
cost_usd (Listpreis)	\$30.47	\$18.80	\$11.20 🏆	\$6.22	\$2.26	\$1.65 🏆
duration_seconds	2530	2230	1647 🏆	627	516	317 🏆
code_mass (APP)	879	827	807 🏆	153	149 🏆	158
cognitive_max	5.0	4.8	4.2 🏆	4.3 🏆	6.2	7.6
refactorings_applied	24.9 🏆	19.0	16.8	7.9 🏆	5.0	3.0

total_tokens ist über Harnesse nicht direkt vergleichbar (CC/OC zählen Cache-Reads aggressiv, pi nicht) — daher cost_usd (Listpreis) und input+output als Effizienz-Proxys.

Befunde:

- **F-harness.1** — Korrektheit ist harness-invariant; CC × claim-office zeigt leichte Streuung
- **F-harness.2** — Token-Footprint und Listpreis-Kosten: pi ist die günstigste Variante
- **F-harness.3** — Code-Mass (APP) ist harness-invariant; mccabe/longest/cognitive variieren kata-abhängig
- **F-harness.4** — Claude-Code-Harness-Glitch: premature end_turn auf claim-office (Thinking-Variante)
- **F-harness.5** — TDD-Disziplin ist harness-invariant; Refactor-Häufigkeit fällt CC → OC → pi monoton ab
- **F-harness.6** — Pi-Cycle-Inflation auf claim-office: deutlich mehr Red-Marker als CC/OC bei gleicher Test-Anzahl

Bei konstantem Modell, Workflow-Intention und Prompt-Stil ist die Korrektheit harness-invariant (fünf von sechs Zellen deterministisch 1.00; CC × claim-office leicht streuend bei 0.96). Auch Code-Mass (APP) und TDD-Disziplin sind harness-invariant — lediglich die Refactor-Häufigkeit fällt monoton CC → OC → pi. Bei den Kosten ist pi nach Listpreis die günstigste und schnellste Variante; cache-bereinigt kehrt sich das Ranking aber um (CC am sparsamsten), weil pi keine Prompt-Cache-Rabatte erhält. Caveat: total_tokens harness-übergreifend nicht vergleichbar; pi-Kosten teilweise durch einen Portkey-Caching-Bug verzerrt; n=5. Details: [research/questions-cross/1.1-harness-effect/findings.md](#).

Workflow-Entwicklung

1.1 RQ-pep-v6.1 — Liefern psychologische Begründungen ('pep talks') in den Red- und Green-Skill-Prompts auf v6.1-Basis einen messbaren Code-Qualitaets- oder TDD-Disziplin-Vorteil ueber rein operationale Anweisungen?

Datenbasis: 10 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=5.

Übersicht (game-of-life-example-mapping; Code-Qualität kleiner = besser)

Outcome	v6.1-hybrid (pep)	v6.1-no-pep
code_mass (APP)	147.6	144.6 🏆
smell_total	2.6	2.0 🏆
cc_longest_function	15.0	13.2 🏆
cognitive_max	6.2	4.6 🏆
mccabe_max	5.2	4.8 🏆

Alle Δ innerhalb ~1 σ — kein klarer Code-Qualitäts-Effekt, Trend leicht pro Reduktion.

Befunde:

- **F-1.1** — Pep-Talks: kein Code-Qualitäts-Vorteil, aber Disziplin-Verschiebung

Das Entfernen psychologischer Begründungen ("pep talks") aus den Red-/Green-Prompts verschlechtert auf game-of-life weder Korrektheit (beide 100 %) noch Code-Qualität. Es verschiebt aber das TDD-Verhalten: die reduzierte Variante refactoriert deutlich häufiger (+67 %) und implementiert seltener vorab (~75 % tests_passed_immediately) — entgegen der a-priori-Hypothese. Der Preis sind +30 % Wallclock und +21 % Tokens durch die zusätzlichen Refactor-Cycles. Caveat: nur game-of-life, n=5, alle Deltas mit großer Streuung. Details: [research/workflow-dev/1.1-pep-effect-v6.1/findings.md](#).

1.2 RQ-emoji-v6.1 — Haben Decoration-Emojis (🟢🔴🟡🟠🟤🟣🟦🟧🟨🟩🟪🟫🟬🟭🟮🟯🟰🟱🟲🟳🟴🟵🟶🟷🟸🟹🟺🟻🟼🟽🟾🟿🟰🟱🟲🟳🟴🟵🟶🟷🟸🟹🟺🟻🟼🟽🟾🟿) in den Workflow-Prompts (Skills + Refactor-Agent + rules/tdd.md) auf v6.1-Basis einen messbaren Effekt auf Code-Qualitaet oder TDD-Disziplin?

Datenbasis: 10 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=5.

Übersicht (game-of-life-example-mapping; Code-Qualität kleiner = besser)

Outcome	v6.1-hybrid (emoji)	v6.1-no-emoji
code_mass (APP)	147.6 🏆	156.8
smell_total	2.6	2.0 🏆
cc_longest_function	15.0	11.4 🏆
cognitive_max	6.2 🏆	6.6
mccabe_max	5.2	4.6 🏆

Trophäen splitten 3:2, jede Differenz < 1 σ — kein direktonaler Code-Qualitäts-Effekt.

Befunde:

- **F-1.1** — Decoration-Emojis: kein Code-Qualitäts-Effekt, leichte Disziplin-Verschiebung
- **F-1.2** — Decoration-Emojis sparen keine Tokens

Das Entfernen aller 95 Decoration-Emojis aus den Workflow-Prompts hat keinen Code-Qualitäts-Effekt (Trophäen splitten 3:2, alle Δ < 1 σ) und keinen Korrektheits-Effekt (beide 100 % innen und außen). Wie bei der Pep-Reduktion verschiebt sich nur das TDD-Verhalten leicht (mehr Refactoring, weniger Sofort-Grün). Entgegen der Erwartung sparen die Emojis keine Tokens — die reduzierte Variante ist sogar minimal teurer (+8.5 %), weil die zusätzlichen Refactor-Phasen kosten. Emoji-Header bleiben als "decorative, safe to drop" klassifiziert. Caveat: nur game-of-life, n=5. Details: [research/workflow-dev/1.2-emoji-effect-v6.1/findings.md](#).

1.3 RQ-pep-emoji-v6.1 — Sind die Effekte der pep- und emoji-Reduktionen auf v6.1-Basis additiv (zwei unabhaengige Kanaele) oder gemeinsam getragen (ein 'Prompt-Drumherum'-Mechanismus)?

Datenbasis: 25 Runs · Coverage: 4/4 Zellen (100 %) bei min_replicates=5.

Übersicht (game-of-life-example-mapping; Code-Qualität kleiner = besser)

Outcome	hybrid (pep+emoji)	no-pep	no-emoji	no-pep-no-emoji
code_mass (APP)	153.7	144.6 🏆	156.8	153.2
smell_total	2.4	2.0 🏆	2.0 🏆	2.2
cc_longest_function	14.2	13.2	11.4 🏆	16.2
cognitive_max	6.5	4.6 🏆	6.6	7.8
mccabe_max	5.2	4.8	4.6 🏆	5.0

Die kombinierte Reduktion gewinnt keine Code-Qualitäts-Metrik; alle $\Delta < 1 \sigma$.

Befunde:

- **F-1.1** — Pep- und Emoji-Reduktion: keine Additivität, sondern Sättigung mit Anti-Effekt
- **F-1.2** — Korrektheit robust gegen alle Reduktionskombinationen
- **F-1.3** — Kombinierte Reduktion läuft schneller mit weniger Refactor-Phasen

Die Effekte von Pep- und Emoji-Reduktion sind nicht additiv. Für `tests_passed_immediately` saturieren beide bei demselben Wert (Pep-Entfernung allein reicht); für `refactorings_applied` kehrt sich der Einzel-Effekt bei kombinierter Anwendung sogar um (kombiniert 3.8 vs. additive Vorhersage ~9.1, sogar unter Baseline). Eine sparsame Lesart: jede Reduktion entfernt eine Reassurance-Schicht, die zum Refactor-Spawn motiviert — fehlen beide, fehlt der Trigger. Korrektheit bleibt über alle vier Kombinationen 100 %; die kombinierte Variante läuft am schnellsten. Caveat: nur game-of-life, n=5, Routing-Asymmetrie in einer Zelle. Details: [research/workflow-dev/1.3-pep-emoji-combined-v6.1/findings.md](https://research.workflow-dev/1.3-pep-emoji-combined-v6.1/findings.md).

1.4 RQ-pep-emoji-claim-office — Haelt der Interaktions-Befund aus RQ-pep-emoji-v6.1 (Pep+Emoji-Reduktion: Anti-Additivitaet bei refactorings_applied, Saettigung bei tests_passed_immediately, Korrektheit invariant) auch auf einer komplexeren Kata mit echten Mehrdeutigkeiten?

Datenbasis: 22 Runs · Coverage: 4/4 Zellen (100 %) bei min_replicates=5.

Übersicht (claim-office-example-mapping; verification_pct höher = besser)

Workflow	n	verification_pct mean	min	std	tests_passing
v6.1-hybrid (pep+emoji)	5	1.00 🏆	1.00	0.00	100% 🏆
v6.1-no-pep	5	0.97	0.87	0.06	100% 🏆
v6.1-no-emoji	5	0.80	0.00	0.45	80%
v6.1-no-pep-no-emoji	5	0.95	0.73	0.12	100% 🏆

Nur die Vollvariante (pep+emoji) liefert reproduzierbar perfekte Korrektheit (std=0); alle Reduktionen verlieren.

Befunde:

- **F-1.1** — Pep- und Emoji-Reduktion brechen Korrektheits-Garantie auf claim-office
- **F-1.2** — Disziplin-Interaktion kehrt sich um vs. game-of-life
- **F-1.3** — Recipe-Empfehlung für komplexe Katas: v6.1-hybrid behalten

Auf der komplexen claim-office-Kata kehrt sich das game-of-life-Bild um: die Vollvariante mit Pep und Emoji ist der einzige Workflow mit reproduzierbar perfekter Korrektheit ($5/5 \times 1.00$). Alle Reduktionen verlieren — die Emoji-Reduktion sogar katastrophal (ein 0/15-Run, bei dem der Agent nach der Test-Liste abbricht). Auch das Disziplin-Muster dreht sich: hier refactoriert die Vollvariante am meisten, die Reduktionen kürzen die Iterationen ab. Lesart: Marker und Pep liefern auf neuem, mehrdeutigem Code Re-Orientierungs-Anker, die auf trainingsbekanntem Code redundant sind. Caveat: n=5; die 0/15-Failure dominiert die Statistik. Details: research/workflow-dev/1.4-pep-emoji-claim-office/findings.md.

1.5 RQ-why-block-effect-v6.1 — Tragen Why-Blocke (kausale Begründungen neben MUSTs) auf v6.1-Basis einen messbaren TDD-Disziplin- oder Korrektheits-Vorteil ueber rein imperative Anweisungen — bei voll erhaltenem PEP?

Datenbasis: 15 Runs · Coverage: 1/2 Zellen (50 %) bei min_replicates=8.

Übersicht (claim-office-example-mapping; Baseline vs. with-why; ↑ höher = besser, ↓ kleiner = besser)

Metrik	Richtung	Baseline	with-why
verification_pct	↑	1.00 🏆	0.91
refactorings_applied	↑	9.88	18.50 🏆
code_mass	↓	840.75	769.12 🏆
smell_total	↓	2.88	0.38 🏆
cc_longest_function	↓	23.38 (max 60)	13.25 🏆 (max 15)
cognitive_max	↓	7.62 (max 21)	4.38 🏆 (max 6)
mccabe_max	↓	6.75	4.25 🏆
duration_seconds	↓	1464 🏆	2234
total_tokens	↓	32.6 M 🏆	39.8 M

Befunde:

- **F-1.1** — Why-Blöcke ohne Korrektheits-Effekt, mit deutlichem Disziplin- und Code-Qualitäts-Effekt
- **F-1.2** — Cost-Trade-off: rund 50 % mehr Wallclock, rund 22 % mehr Tokens — pro Cycle aber gleich schnell

Kausale Why-Blöcke neben den MUSTs ("MUST X. Why: Y.") haben keinen Korrektheits-Effekt (1.00 vs. 0.91, der Unterschied geht auf einen 0-Cycle-Outlier zurück), aber einen deutlichen, gleichgerichteten Effekt auf Disziplin und Code-Qualität: +87 % Refactorings, -87 % Smells, `cognitive_max` -43 %, und — besonders auffällig — eine Streuungs-Reduktion von 82–90 % auf allen Komplexitäts-Spitzen. Der Preis sind ~50 % mehr Wallclock und ~22 % mehr Tokens, getrieben durch mehr Cycles (pro Cycle ist with-why sogar leicht token-effizienter). Caveat: nur claim-office, nur opus-4-7-portkey-no-thinking, Coverage 1/2 Zellen. Details: [research/workflow-dev/1.5-why-block-effect-v6.1/findings.md](#).

1.6 RQ-v62-cleanup-validation-v61-with-why — Veraendern die drei v6.5.1-Audit-Cleanups (Konsistenz, refactor.md-Entkopplung, tdd-experiment-mode-Reframing) — angewendet auf v6.1-with-why — messbar das Workflow-Verhalten auf claim-office, oder ist v6.2-with-why-cleaned eine verhaltens-aequivalente Hygiene-Variante der neuen Default-Baseline?

Datenbasis: 16 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=8.

Übersicht (claim-office-example-mapping; ↑ höher = besser, ↓ kleiner = besser)

Metrik	Richtung	v6.1-with-why	v6.2-with-why-cleaned
verification_pct mean	↑	0.91	0.96 🏆
refactorings_applied mean	↑	18.50	24.88 🏆
code_mass mean	↓	769.12 🏆	878.50
smell_total mean	↓	0.38 🏆	0.38 🏆
cc_longest_function mean	↓	13.25	12.38 🏆
cognitive_max mean	↓	4.38 🏆	5.00
mccabe_max mean	↓	4.25 🏆	4.50
duration_seconds mean	↓	2234 🏆	2530
total_tokens mean	↓	39.78 M 🏆	44.44 M

Befunde:

- **F-1.1** — Cleanups sind verhaltens-äquivalent zur Baseline; keine Korrektheits-Regression
- **F-1.2** — Disziplin-Drift: mehr Refactorings, engere Streuung
- **F-1.3** — Code-Qualität neutral bis leicht verschlechtert; Smells unverändert
- **F-1.4** — Kosten-Aufschlag durch Mehr-Refactorings: +13 % Wallclock, +12 % Tokens

Die drei Audit-Cleanups (Konsistenz-Renames, refactor.md-Entkopplung, Mode-Reframing) sind auf claim-office verhaltens-äquivalent zur Baseline — keine Korrektheits-Regression (0.91 → 0.96, Streuung von σ 0.26 auf 0.09 gekollabiert). Sie erzeugen eine Disziplin-Drift (+34 % Refactorings, kein Refactoring-Aussetzer mehr) bei neutraler bis minimal verschlechterter Code-Qualität (Smells unverändert 0.38). Der Preis sind +13 % Wallclock und +12 % Tokens. Die Variante wird damit zur neuen Default-Baseline für korrektkeits-kritische Arbeit. Caveat: n=8, eine Kata; die Korrektheits-Differenz ist im Noise nicht von Null zu trennen. Details: [research/workflow-dev/1.6-v62-cleanup-validation-v61-with-why/findings.md](#).

1.7 RQ-v62-cleanup-validation-gol — Generalisiert das Cleanup-Aequivalenz-Ergebnis aus RQ-1.6 (claim-office) auch auf die trainings-bekannte game-of-life-Kata, oder zeigt v6.2-with-why-cleaned dort einen anderen Effekt als auf claim-office?

Datenbasis: 15 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=5.

Übersicht (game-of-life-example-mapping; ↑ höher = besser, ↓ kleiner = besser)

Metrik	Richtung	v6.1-with-why	v6.2-with-why-cleaned
predictions_correct_rate	↑	98.8 %	100 % 🏆
refactorings_applied mean	↑	6.40	7.80 🏆
code_mass mean	↓	151.0	148.8 🏆
smell_total mean	↓	2.80	2.60 🏆
cc_longest_function mean	↓	9.40 🏆	9.40 🏆
cognitive_max mean	↓	4.80 (max 15)	2.80 🏆 (max 4)
mccabe_max mean	↓	4.60	3.60 🏆
duration_seconds mean	↓	569 🏆	644
total_tokens mean	↓	7.56 M 🏆	8.67 M

Befunde:

- **F-1.1** — Cleanup-Aequivalenz generalisiert: keine Korrektheits-Regression auf GoL
- **F-1.2** — Spitzen-Komplexitaet kollabiert: `cognitive_max` -42 %, `mccabe_max` -22 %, Streuung stark gekappt
- **F-1.3** — Disziplin-Drift uebertraegt sich auf GoL: +22 % Refactorings (claim-office: +34 %)
- **F-1.4** — Kosten-Aufschlag konsistent: +13 % Wallclock, +15 % Tokens

Das Cleanup-Äquivalenz-Ergebnis generalisiert auf die trainingsbekannte game-of-life-Kata: Korrektheit invariant (beide 100 %), und die Spitzen-Komplexität sinkt sogar deutlich (`cognitive_max` -42 %, `mccabe_max` -22 %, Streuung -64–81 %). Die Disziplin-Drift überträgt sich (+22 % Refactorings, gegenüber +34 % auf claim-office) bei gleichem Kosten-Aufschlag (+13 % Wallclock, +15 % Tokens). Der Streuungs-Kollaps auf den Komplexitäts-Spitzen ist ein wiederkehrendes Muster (mehr Refactorings → konsistentere Spitzen), kata- und iterations-unabhängig. Caveat: n=5 (bewusst kleiner als die claim-office-Validierung). Details: [research/workflow-dev/1.7-v62-cleanup-validation-gol/findings.md](#).

1.8 RQ-audit-bundle-v62 — Reproduziert das Audit-Bundle (Rationale-Ergaenzungen + Red-Phase-Hardening) auf der v6.2-with-why-cleaned-Basis die in der archivierten RQ-audit gegen v6.5-lean gemessenen Effekte (Disziplin-Plus, Streuungs-Schrumpf, Token/Wallclock-Aufschlag bei Korrektheits-Erhalt)?

Datenbasis: 20 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=10.

Übersicht (game-of-life-example-mapping, n=10 je Workflow; Richtung je Outcome notiert)

Outcome	v6.2-with-why-cleaned	v6.3-audit-bundle
tests_passed_immediately (kleiner = besser)	0.7	0 🏆
refactorings_applied (höher = besser)	7.9	8.7 🏆
predictions_correct_rate (höher, pooled)	100 % 🏆	97.4 %
code_mass (kleiner = besser)	153.3	149.3 🏆
smell_total (kleiner = besser)	2.4	2.2 🏆
cognitive_max (kleiner = besser)	4.3	4.5
tests_passing / completed_within_budget	100 % / 100 %	100 % / 100 %
total_tokens (kleiner = besser)	8.32 M 🏆	9.65 M

Befunde:

- **F-1.8.1** — Mandatory-Procedure-Preamble eliminiert vorzeitige Greens auch auf v6.2-Basis
- **F-1.8.2** — Refactor-Rationale + Drei-Pfad-Bar erhöht und stabilisiert Refactoring-Disziplin
- **F-1.8.3** — Code-Qualität gleichwertig, kein Regress
- **F-1.8.4** — Wrong-Predictions-Block zeigt sich in 2.6 pp Predictions-Rate-Drop
- **F-1.8.5** — Audit-Bundle kostet +16 % Tokens, aber kein Wallclock-Aufschlag
- **F-1.8.6** — Streuungs-Schrumpf bestätigt sich in Disziplin- und Komplexitäts-Outcomes

Auf game-of-life reproduziert das Audit-Bundle (Rationale-Ergänzungen + Red-Phase-Hardening) die erwarteten Effekte: vorzeitige Greens fallen auf 0, die Refactor-Disziplin steigt und stabilisiert sich (σ auf ein Drittel), die Code-Qualität bleibt gleichwertig. Der predictions_correct_rate-Drop (100 % → 97.4 %) ist intendiert — das Backfill-Verbot lässt ehrliche Falsch-Vorhersagen stehen. Der Preis sind +16 % Tokens ohne Wallclock-Aufschlag. Caveat: nur game-of-life, n=10 — auf der novel Kata kippt das Bild (siehe RQ-1.9). Details: [research/workflow-dev/1.8-audit-bundle-effect-v62/findings.md](https://research.workflow-dev/1.8-audit-bundle-effect-v62/findings.md).

1.9 RQ-audit-bundle-claim-office — Generalisiert der RQ-1.8-Befund (Audit-Bundle wirkt Disziplin-stabilisierend und Code-Qualitäts-neutral auf v6.2-Basis × game-of-life) auch auf die novel claim-office-Kata, oder kippt das Pattern dort wie schon in RQ-1.4 fuer Reduktionen geschehen?

Datenbasis: 16 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=8.

Übersicht (claim-office-example-mapping, n=8 je Workflow; viele v6.3-Werte durch frühen Run-Abbruch verzerrt)

Outcome	v6.2-with-why-cleaned	v6.3-audit-bundle
verification_pct (höher = besser)	0.96 🏆	0.35 (bi-modal)
cycle_count (höher = vollständiger)	37.4 🏆	11.3
code_mass (durch frühen Stopp irreführend)	878.5	441.5
smell_total (kleiner = besser)	0.38 🏆	0.50
total_tokens (durch frühen Stopp niedriger)	44.4 M	16.7 M

Befunde:

- **F-1.9.1** — Audit-Bundle bricht Korrektheit auf claim-office von 96 % auf 35 %
- **F-1.9.2** — Bi-modale Vollständigkeit: 6 von 8 Runs stoppen vorzeitig
- **F-1.9.3** — Mandatory-Preamble eliminiert vorzeitige Greens, aber Effekt-Größe irrelevant durch frühen Stopp
- **F-1.9.4** — Code-Qualitäts-Metriken niedriger, aber durch Unvollständigkeit verzerrt
- **F-1.9.5** — Empfehlung: v6.3 nicht als Default-Baseline für claim-office promoten

Der RQ-1.8-Befund generalisiert **nicht** auf die novel Kata: dasselbe Audit-Bundle bricht hier die Korrektheit von 0.96 auf 0.35 ein (6 von 8 Runs stoppen vorzeitig, experiment-done.txt fehlt). Der Bruch ist kein "falsche Implementation", sondern "unvollständige Implementation" — intern grün, extern kollabiert. Damit bleibt das Audit-Bundle game-of-life-Champion ohne claim-office-Default-Status. Das Muster deckt sich mit den Reduktions-Brüchen aus RQ-1.4 und RQ-1.10. Caveat: n=8; die niedrigeren Komplexitäts-Werte sind Unvollständigkeits-Artefakte, kein Qualitäts-Gewinn. Details: [research/workflow-dev/1.9-audit-bundle-validation-claim-office/findings.md](https://research.workflow-dev/1.9-audit-bundle-validation-claim-office/findings.md).

1.10 RQ-refactor-vocab-v62 — Verbessert ein additiver Vokabular-Block im refactor-Agent (Cyclomatic + Cognitive Complexity, Single Responsibility, Smell → Move-Tabelle) die Code-Qualität auf v6.2-with-why-cleaned-Basis, ohne Korrektheit oder Kosten signifikant zu beeinträchtigen?

Datenbasis: 28 Runs · Coverage: 4/4 Zellen (100 %) bei min_replicates=5.

Übersicht (v6.2.1-refactor-vocab vs. Baseline; Bundle-Risiko-Sentinel code_mass = diagnostisch für "Agent hat aufgehört")

Kata	Metrik	v6.2-with-why-cleaned	v6.2.1-refactor-vocab
game-of-life	tests_passing-Rate	100 % 🏆	100 % 🏆
game-of-life	code_mass (mean)	153 🏆	172
claim-office	verification_pct (mean)	0.96 🏆	0.23
claim-office	verification_pct (min)	0.73 🏆	0.00
claim-office	cycle_count (mean)	37.4 🏆	16.0
claim-office	code_mass (mean)	879 🏆	372

Befunde:

- **F-1.10.1** — Korrektheits-Kollaps auf claim-office (Bundle-Risiko bestätigt)
- **F-1.10.2** — game-of-life: kein erkennbarer Code-Qualitäts-Gewinn
- **F-1.10.3** — TDD-Disziplin: GoL neutral, claim-office gestört (Folge von F-1.10.1)

Der additive Refactor-Vokabular-Block (Complexity-Awareness + SRP + Smell-Tabelle) bricht die Korrektheit auf claim-office von 0.96 auf 0.23 — in 4/5 Runs terminiert der Agent vorzeitig (cycle_count und code_mass halbiert, kein Rate-Limit/Timeout). Auf game-of-life bringt der Block keinen messbaren Code-Qualitäts-Gewinn (alle Komplexitäts-Deltas < 1 σ). Damit reproduziert sich exakt das Bundle-Bruch-Muster aus RQ-1.9; die Variante wird nicht promotet. Caveat: claim-office-Zelle n=5, ein 0.93-Run unter sonst kollabierenden Runs. Details: [research/workflow-dev/1.10-refactor-vocab-effect-v62/findings.md](#).

1.11 RQ-metric-driven-refactor-v62 — Verbessert ein Refactor-Agent, der deterministische Metriken (ESLint smells, SonarJS cognitive complexity, McCabe cyclomatic complexity) selbst pre/post misst und APP-Mass parallel ausweist, die Code-Qualitaet auf claim-office gegenueber dem Baseline-v6.2-with-why-cleaned-Workflow — ohne Korrektheit oder TDD-Disziplin zu beschadigen?

Datenbasis: 13 Runs · Coverage: 2/2 Zellen (100 %) bei min_replicates=5.

Übersicht (claim-office-example-mapping, opus-4-7-no-thinking; Richtung je Outcome notiert)

Outcome	v6.2-with-why-cleaned (n=8)	v6.4-metric-driven-refactor (n=5)
verification_pct (höher)	0.96	0.99 🏆
cognitive_max (kleiner)	5.0	2.4 🏆
mccabe_max (kleiner)	4.5	3.0 🏆
code_mass (kleiner)	879	805 🏆
smell_total (kleiner)	0.38	0.0 🏆
cc_longest_function (kleiner)	12.4 🏆	13.0
refactorings_applied (höher)	24.9	30.4 🏆
duration_seconds (kleiner)	2530 🏆	5284
total_tokens (kleiner)	44.4 M 🏆	102.3 M

Befunde:

- **F-1.1** — Spitzen-Komplexität halbiert ohne Korrektheitskosten
- **F-1.2** — Mehr und engmaschigere Refactor-Zyklen, weniger out-of-the-box-Greens
- **F-1.3** — Bundle-Bruch aus RQ-1.9 und RQ-1.10 nicht reproduziert
- **F-1.4** — Kosten-Aufschlag stark und sehr volatil
- **F-1.5** — Predictions-Rate sinkt durch ehrlichere Falschvorhersagen, nicht Format-Bruch

Ein Refactor-Agent, der ESLint-Smells, SonarJS-Cognitive- und McCabe-Komplexität selbst pre/post misst, halbiert die Spitzen-Komplexität auf claim-office (cognitive_max 5.0 → 2.4, smell_total 0.38 → 0) und macht die Korrektheit sogar robuster (0.96 → 0.99). Entscheidend: der Bundle-Bruch aus RQ-1.9/1.10 tritt **nicht** auf — alle 5 Runs durchlaufen volle TDD-Loops mit experiment-done.txt. Die Per-Cycle-Aufwands-Erhöhung über deterministische Tools triggert den Self-Stop also nicht, anders als semantische Erweiterungen. Der Preis ist hoch und volatil: +130 % Tokens, +109 % Wallclock. Caveat: n=5, eine Kata, ein Modell. Details: [research/workflow-dev/1.11-metric-driven-refactor-effect-v62/findings.md](#).

1.12 RQ-end-refactor-v62 — Verbessert ein metric-driven Refactor-Pass die Code-Qualitaet gegenueber dem Per-Cycle-Baseline-Workflow (v6.2-with-why-cleaned) — und greift der Hebel als rein per-cycle (v6.4-metric-driven-refactor) oder als zusaetzlicher Whole-src-End-Pass (v6.5-end-refactor) — ohne Korrektheit oder TDD-Disziplin zu beschadigen, und haelt der Befund ueber zwei Kata-Typen (mehnteilige CLI-Codebasis claim-office vs einteilige Library game-of-life)?

Datenbasis: 43 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Übersicht: cognitive_max (kleiner = besser) — opus-4-7, nie über Katas gemittelt; 🏆 = bester Wert je Kata (Spread ≥ 1 σ)

Kata	v6.2 (Baseline)	v6.4 (per-cycle)	v6.5 (end-refactor)
claim-office	5.0	2.4 🏆	2.8
game-of-life	4.0	2.2 🏆	3.0

Befunde:

- **F-1.12.1** — Auf beiden Katas ist der per-cycle-Refactor v6.4 der robuste Spitzen-Komplexitäts-Sieger
- **F-1.12.2** — Der End-Refactor v6.5 wirkt nur auf der mehrteiligen Codebasis; auf der einteiligen Library ist er Rauschen
- **F-1.12.3** — Korrektheit und TDD-Disziplin überall intakt; kein Bundle-Bruch

- **F-1.12.4** — Kosten steigen monoton mit der Refactor-Intensität; per-cycle ist auf großen Codebasen kosten-unvorhersehbar
- **F-1.12.5** — Keine globale v6.5-Baseline-Promotion; metric-driven Refactor lohnt, der wirksame Hebel-Zeitpunkt ist kata-abhängig

Beide metric-driven Refactor-Varianten schlagen die Per-Cycle-Baseline bei der Spitzen-Komplexität, ohne die Korrektheit zu brechen (claim-office ≥ 0.96 , game-of-life 1.0). Auf Opus 4.7 ist der **Per-Cycle-Refactor** der robuste Komplexitäts-Sieger über beide Kata-Typen; der einmalige **End-Pass** wirkt nur auf der mehrteiligen CLI-Codebasis (Cross-file-Konsolidierung, kleinste Code-Mass) und ist auf der einteiligen Library Rauschen. Kein Arm ist globaler Sieger — die Empfehlung ist kata-abhängig. Caveat: claim-office-Vergleich teils routing-konfundiert (v6.4 native, v6.2/v6.5 portkey). Details: [research/workflow-dev/1.12-end-refactor-effect-v62/findings.md](#).

1.13 RQ-end-refactor-opus48 — Haelt der v6.5-end-refactor-Befund aus RQ-1.12 (Korrektheit intakt, Code-Qualitaet $\geq$ v6.2, Token-Kosten $\sim$ v6.2) auf Opus 4.8 (no-thinking) — oder taeuscht der zusaetzliche End-Refactor-Pass auf dem neuen Modell die claim-office-Vollstaendigkeit aus (Bundle-Bruch-Muster aus RQ-1.9/RQ-1.10)?

Datenbasis: 30 Runs · Coverage: 6/6 Zellen (100 %) bei min_replicates=5.

Übersicht: **cognitive_max (kleiner = besser)** — opus-4-8-no-thinking, nie über Katas gemittelt; 🏆 = bester Wert je Kata (Spread $\geq 1 \sigma$)

Kata	v6.2 (Baseline)	v6.4 (per-cycle)	v6.5 (end-refactor)
claim-office	3.6	3.6	2.8 🏆
game-of-life	5.6	3.2	2.4 🏆

Befunde:

- **F-1.13.1** — v6.5-end-refactor hält die Korrektheit auf Opus 4.8; RQ-1.12-Kernbefund repliziert
- **F-1.13.2** — Auf Opus 4.8 ist die nackte v6.2-Baseline weniger robust als v6.4/v6.5; ein CLI-Vertragsbruch durch Workflow-Umgehung
- **F-1.13.3** — Metric-driven Refactor lohnt auf beiden Katas; v6.5 führt bei Spitzen-Komplexität, sonst gleichauf — kein Workflow strikt besser
- **F-1.13.4** — Kosten kata-abhängig: auf claim-office alle drei innerhalb σ , auf game-of-life monoton steigend
- **F-1.13.5** — TDD-Disziplin auf 4.8 für alle drei Workflows intakt, auf beiden Katas
- **F-1.13.6** — Keine globale v6.5-Promotion auf 4.8; die F-1.12.2-Kata-Asymmetrie repliziert nur teilweise

Auf Opus 4.8 hält der End-Refactor-Pass die Korrektheit auf beiden Katas (5/5 perfekt) — der RQ-1.12-Kernbefund repliziert, das Bundle-Bruch-Muster tritt nicht auf. Die Rangordnung dreht sich aber: auf 4.8 führt der **End-Pass** (v6.5) bei der Spitzen-Komplexität auf beiden Katas, während auf 4.7 der Per-Cycle-Refactor der Sieger war — der wirksame Refactor-Hebel ist also modellabhängig. Bemerkenswert: die nackte Baseline ist auf 4.8 am wenigsten robust (1/5 CLI-Vertragsbruch durch bewusste Workflow-Umgehung). Caveat: n=5, gegenüber RQ-1.12 routing- und modell-konfundiert. Details: [research/workflow-dev/1.13-end-refactor-opus48/findings.md](#).

5. Cross-RQ-Synthese

1. **Korrektheits-Effekte werden ausschließlich auf novel Katas sichtbar — und dann sind sie groß.** Prompt-Stil (RQ-prompt-correctness: +64 pp durch Example-Mapping) und Test-Phase (RQ-tdd-quality: Vibe 0.28 $\rightarrow$ ≥ 0.96) sind zwei unabhängige, starke Korrektheits-Hebel auf claim-office. Auf der trainingsbekannten game-of-life-Kata verschwinden beide vollständig (RQ-prompt-known-kata, RQ-stability: alle Stile/Workflows bei 1.00). Wer Korrektheit messen will, braucht eine Kata, die das Modell nicht memoriert hat.
2. **Die game-of-life/claim-office-Asymmetrie ist das durchgängige methodische Leitmotiv der Workflow-Entwicklung.** Vier unabhängige Workflow-Änderungen sind auf der trainingsbekannten Kata neutral bis positiv, brechen aber auf der novel Kata die Korrektheit ein: Pep-/Emoji-Reduktion (RQ-1.4), Audit-Bundle (RQ-1.8 vs. RQ-1.9: 0.96 $\rightarrow$ 0.35), Refactor-Vokabular (RQ-1.10: 0.96 $\rightarrow$ 0.23). Nur die deterministische Tool-Messung (RQ-1.11) entkommt dem Muster. Das verbindet RQ-1.4/1.8/1.9/1.10/1.11 zu einer Regel, die in keiner einzelnen RQ steht: additive *semantische* Prompt-Erweiterungen tragen ein Self-Stop-Risiko, das nur eine novel Kata aufdeckt.
3. **Der Code-Qualitäts-Hebel ist Refactor-Isolation und Refactor-Häufigkeit, nicht Phasen-Isolation an sich.** RQ-context (nur der isolierte Refactor-Subagent senkt die Komplexität, zusätzliche Green-Isolation nicht), RQ-why-block (mehr Refactorings $\rightarrow$ Streuungs-Kollaps der Komplexitäts-Spitzen) und RQ-stability (der Hybrid hat 0 % Outlier-Rate) zeigen denselben Mechanismus aus drei Richtungen. Volle Phasen-Isolation schadet auf der komplexen Kata sogar.
4. **Workflow und Modell sind nicht separabel — “bester Workflow” ist immer eine modell-bedingte Aussage.** RQ-workflow-model (v4 und v6 tauschen je nach Modell die Plätze), RQ-model-novel (die beiden Modell-Ränder schlagen das mittlere) und RQ-1.12 vs. RQ-1.13 (der wirksame Refactor-Zeitpunkt ist auf Opus 4.7 der Per-Cycle-Pass, auf Opus 4.8 der End-Pass) ergeben zusammen: jede Workflow-Empfehlung muss pro Modell neu validiert werden.
5. **Korrektheit ist zunehmend gelöst; die offenen Achsen sind Kosten und Code-Qualität.** Über RQ-tdd-correctness, RQ-stability und RQ-harness erreichen die meisten Setups ~ 100 % Korrektheit, während die echten Spreads in Tokens (Faktor ~ 16 in RQ-tdd-quality) und Komplexität liegen. Der harness-übergreifende Modell-Vergleich (RQ-model-quality-oc/RQ-model-novel-oc) zeigt zudem, dass mehrere Nicht-Anthropic-Modelle dieselbe Korrektheit zu einem Bruchteil der Kosten liefern — die Kosten-Achse ist damit zunehmend eine Modell-Wahl, nicht nur eine Workflow-Wahl.

6. Limitierungen

- **Nur Anthropic-Modelle im Kern.** Der Claude-Code-Kern nutzt ausschließlich Opus/Sonnet/Haiku. Die Nicht-Anthropic-Modelle (GLM, Gemini, Kimi, DeepSeek, MiniMax, Mistral) erscheinen nur in den explizit gekennzeichneten OpenCode-RQs — ihre Befunde sind nicht auf den Claude-Code-Harness übertragbar.

- **Nur zwei synthetische Carrier-Katas.** Code-Qualität trägt game-of-life (Library, trainingsbekannt), Korrektheit trägt claim-office (CLI, novel). mars-rover als dritte Achse ist angelegt, aber kaum bespielt. Reale Codebasen (Web-Apps, Datenbank-Code, Async-Systeme) sind nicht abgedeckt.
- **Nur TypeScript** mit einem festen pnpm/tsx/Vitest/ESLint+SonarJS-Stack. Transfer auf Python, Go, Java ist offen.
- **Headless, ohne Human-in-the-Loop.** Alle Runs laufen vollautonom via `claude --print`. HITL-Workflows (Mensch reviewed pro Cycle, IDE-Integration, Pair-Programming) sind außerhalb des Scopes — gerade die Bundle-Bruch-Befunde (Self-Termination) wären unter HITL durch Re-Trigger reparierbar.
- **Kleine Replikat-Zahlen.** Die meisten Zellen haben $n = 3-10$; RQ-stability zeigt, dass $n=3$ die volle Workflow-Rangordnung nur in 25–60 % der Fälle rekonstruiert und Tail-Quantile (Token/Wallclock-Ausreißer) erst ab $n \geq 30$ belastbar sind. Effektgrößen-Richtungen sind robust, präzise σ -Vergleiche und Mittelfeld-Ränge nicht durchweg.
- **Konkrete Coverage-Lücken.** RQ-prompt-correctness ist bei 83 % (20/24 Zellen, Opus-4.7-Zelle $n=3$ vorläufig); RQ-why-block-effect bei 50 % (1/2 Zellen); RQ-stability bei 83 % (5/6) trotz hohem `min_replicates=10`. RQ-pocock-vs-v62 stützt sich auf nur $n=3$ in der Pocock-Zelle.
- **Routing- und Modell-Konfundierung in Querschnitten.** Einige claim-office-Workflow-Vergleiche mischen Direct-API und Portkey-Routing (als dasselbe Modell behandelt, Kosten getrennt); Cross-RQ-Absolutwert-Vergleiche (z.B. Opus 4.7 vs. 4.8) sind zusätzlich modell-konfundiert und nur als Kontext zu lesen.
- **Goodhart-Risiko bei benannten Metriken.** Workflows, die `cognitive_max/mccabe_max` im Refactor-Prompt benennen (RQ-1.10/1.11), machen diese Metriken zu Compliance-Zielen — Cross-Workflow-Vergleiche auf genau diesen Metriken sind dann asymmetrisch zugunsten der benennenden Variante gefärbt. `mutation_score` als versteckte Metrik ist punktuell erhoben.

7. Reproduzierbarkeit

Alle Daten und Analyse-Skripte liegen im Repo:

- `research/questions-{claude,opencode,cross}/*/README.md` und `research/workflow-dev/*/README.md` — RQ-Definitionen (Frontmatter mit factors/controls/outcomes)
- `research/{questions-claude,questions-opencode,questions-cross,workflow-dev}/*/findings.md` — persistente Befund-Listen
- `experiments/runs/*/metrics.json` — Rohdaten pro Run
- `experiments/aggregate-by-query.py` — RQ-spezifische Aggregation
- `experiments/batch-plan-from-rq.py` — Batch-Plan-Generierung aus RQ-Frontmatter
- `experiments/docker/Dockerfile` + `run-batch.sh` + `batch.sh` — Container-Pipeline
- `experiments/analyze-run.sh` + `analyze_transcript.py` — Run-Analyse

Container-Pin: `claude-code@2.1.156` (siehe `experiments/docker/Dockerfile`; gegenüber dem vorigen Snapshot von 2.1.107 angehoben, weil Opus 4.8 in 2.1.107 nicht verfügbar war).

8. Files

Pfad	Inhalt
<code>research/questions-claude/1.1-prompt-style-correctness/findings.md</code>	RQ-prompt-correctness — Steigert Example-Mapping die Korrektheit gegenüber Prose und User-Story — und ist der Effekt modellabhängig?
<code>research/questions-claude/1.1-prompt-style-correctness/runs.csv</code>	RQ-prompt-correctness aggregierte Run-Metriken
<code>research/questions-claude/1.2-prompt-style-known-kata/findings.md</code>	RQ-prompt-known-kata — Beeinflusst der Prompt-Stil (prose/user-story/example-mapping) bei einer trainingsbekannten Kata (Game of Life) Korrektheit und Code-Qualität — und ist dieser Effekt modellabhängig?
<code>research/questions-claude/1.2-prompt-style-known-kata/runs.csv</code>	RQ-prompt-known-kata aggregierte Run-Metriken
<code>research/questions-claude/2.1-model-effect-code-quality/findings.md</code>	RQ-model-quality — Wie stark unterscheiden sich die verfügbaren Modelle (Sonnet 4.6, Opus 4.6, Opus 4.7, Opus 4.8 — jeweils mit/ohne Thinking) in der Code-Qualität auf einer trainingsbekannten Kata bei stärkstem Workflow?
<code>research/questions-claude/2.1-model-effect-code-quality/runs.csv</code>	RQ-model-quality aggregierte Run-Metriken
<code>research/questions-claude/2.2-model-effect-novel-kata/findings.md</code>	RQ-model-novel — Wie unterscheiden sich Opus 4.8, Opus 4.7 und Opus 4.6 (jeweils no-thinking) in Korrektheit und Code-Qualität auf einer novel Kata mit Mehrdeutigkeiten, die stärker differenziert als die trainingsbekannte game-of-life?
<code>research/questions-claude/2.2-model-effect-novel-kata/runs.csv</code>	RQ-model-novel aggregierte Run-Metriken
<code>research/questions-claude/3.1-workflow-model-interaction/findings.md</code>	RQ-workflow-model — Hängt die Güte eines TDD-Workflows vom Modell ab — gibt es einen universell besten Workflow, oder tauschen verschiedene Workflows je nach Modell die Plätze?
<code>research/questions-claude/3.1-workflow-model-interaction/runs.csv</code>	RQ-workflow-model aggregierte Run-Metriken
<code>research/questions-claude/4.1-tdd-effect-code-quality/findings.md</code>	RQ-tdd-quality — Wie wirkt sich die Workflow-Struktur (von oneshot ueber iterativ bis zu striktem TDD mit Subagents) auf die Code-Qualitaet aus, und macht die TDD-Striktheit einen Unterschied?
<code>research/questions-claude/4.1-tdd-effect-code-quality/runs.csv</code>	RQ-tdd-quality aggregierte Run-Metriken
<code>research/questions-claude/4.2-tdd-effect-correctness/findings.md</code>	RQ-tdd-correctness — Unterscheidet sich die externe Korrektheit (verification_pct) zwischen TDD-Workflow-Varianten auf der neuartigen claim-office-Kata?
<code>research/questions-claude/4.2-tdd-effect-correctness/runs.csv</code>	RQ-tdd-correctness aggregierte Run-Metriken

Pfad	Inhalt
research/workflow-dev/1.10-refactor-vocab-effect-v62/findings.md	RQ-refactor-vocab-v62 — Verbessert ein additiver Vokabular-Block im refactor-Agent (Cyclomatic + Cognitive Complexity, Single Responsibility, Smell → Move-Tabelle) die Code-Qualitaet auf v6.2-with-why-cleaned-Basis, ohne Korrektheit oder Kosten signifikant zu beeintraehtigen?
research/workflow-dev/1.10-refactor-vocab-effect-v62/runs.csv	RQ-refactor-vocab-v62 aggregierte Run-Metriken
research/workflow-dev/1.11-metric-driven-refactor-effect-v62/findings.md	RQ-metric-driven-refactor-v62 — Verbessert ein Refactor-Agent, der deterministische Metriken (ESLint smells, SonarJS cognitive complexity, McCabe cyclomatic complexity) selbst pre/post misst und APP-Mass parallel ausweist, die Code-Qualitaet auf claim-office gegenueber dem Baseline-v6.2-with-why-cleaned-Workflow — ohne Korrektheit oder TDD-Disziplin zu beschaedigen?
research/workflow-dev/1.11-metric-driven-refactor-effect-v62/runs.csv	RQ-metric-driven-refactor-v62 aggregierte Run-Metriken
research/workflow-dev/1.12-end-refactor-effect-v62/findings.md	RQ-end-refactor-v62 — Verbessert ein metric-driven Refactor-Pass die Code-Qualitaet gegenueber dem Per-Cycle-Baseline-Workflow (v6.2-with-why-cleaned) — und greift der Hebel als rein per-cycle (v6.4-metric-driven-refactor) oder als zusaetzlicher Whole-src-End-Pass (v6.5-end-refactor) — ohne Korrektheit oder TDD-Disziplin zu beschaedigen, und haelt der Befund ueber zwei Kata-Typen (mehnteilige CLI-Codebasis claim-office vs einteilige Library game-of-life)?
research/workflow-dev/1.12-end-refactor-effect-v62/runs.csv	RQ-end-refactor-v62 aggregierte Run-Metriken
research/workflow-dev/1.13-end-refactor-opus48/findings.md	RQ-end-refactor-opus48 — Haelt der v6.5-end-refactor-Befund aus RQ-1.12 (Korrektheit intakt, Code-Qualitaet >= v6.2, Token-Kosten ~v6.2) auf Opus 4.8 (no-thinking) — oder taeuscht der zusaetzliche End-Refactor-Pass auf dem neuen Modell die claim-office-Vollstaendigkeit aus (Bundle-Bruch-Muster aus RQ-1.9/RQ-1.10)?
research/workflow-dev/1.13-end-refactor-opus48/runs.csv	RQ-end-refactor-opus48 aggregierte Run-Metriken
experiments/runs/	Alle Run-Verzeichnisse mit Source, Transcript, Metriken